

Web Ontology Language

Elham

Some tutorial

- <https://atomgraph.github.io/SPARQL-Playground/>
- https://docs.data.world/tutorials/sparql/Your_First_Sparql_Query.html

Python packages

- <https://owlready2.readthedocs.io/en/latest/intro.html>
- RDFLib : <https://rdflib.readthedocs.io/en/stable/index.html>
- Read: <https://www.oclc.org/developer/news/2016/making-sense-of-linked-data-with-python.en.html>

- **python ./ontology_viz.py -o test.dot test.ttl -O ontology.ttl** [WARNING] Class `http://dig.isi.edu/Time_Span` doesn't exist in the ontology! [WARNING] Property `http://dig.isi.edu/at_some_time_within_date` doesn't exist in the ontology!
- **dot -Tsvg -o test-ontology-visualization.svg test.dot**
- <https://www.w3.org/2018/09/rdf-data-viz/>

Useful to read

- Database design basics:
- <https://support.microsoft.com/en-gb/office/database-design-basics-eb2159cf-1e30-401a-8084-bd4f9c9ca1f5>
- From SQL Tables To Sparql Graphs ()
- **RDF grapher is a web service for parsing RDF data and visualizing it as a graph:** <https://www.ldf.fi/service/rdf-grapher>
- Stardog: <https://docs.stardog.com/tutorials/rdf-graph-data-model>

- <https://raghavam.github.io/teaching/monsoon2018/ontmodelling/intro-owl.html#/15>

- <https://github.com/Stichting-CROW/ldp-queries/tree/main>
- <https://github.com/ddaedalus/knowledge-systems-ntua/blob/master/ontology/myontology-turtle.owl>

Define ontology

- An explicit specification of a conceptualization(1993-Tom Gruber)
- A formal explicit specification of a shared conceptualization(1997)
- That means an ontology is a machine-readable and precise representation of some subject matter that derived from consensus.
- An ontology as an agreed blueprint for representing knowledge.
- Owl the most widely used ontology representation language.

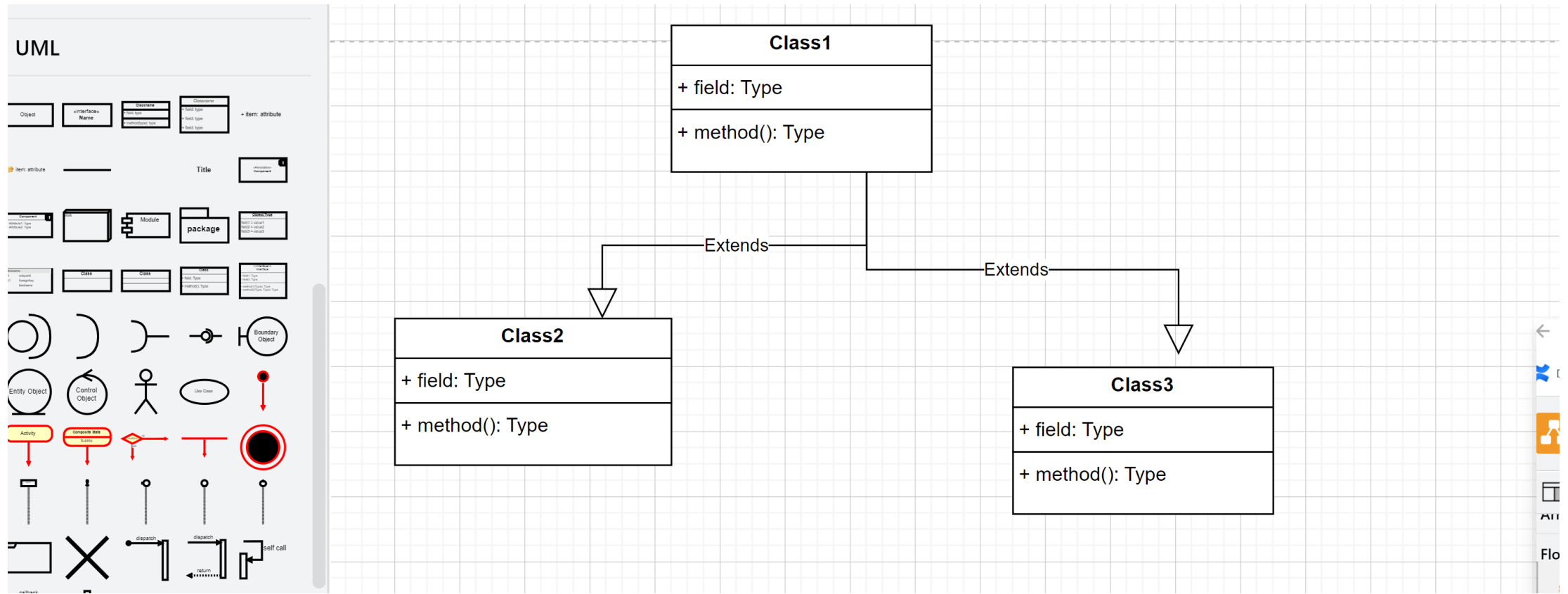
Some example of various published ontologies

- Suggested Upper Merged Ontology (SUMO)
- Descriptive Ontology for Linguistic and Cognitive Engineering(DOLCE)
- gist is Semantic Arts' minimalist upper ontology for the enterprise.
- Basic Formal Ontology (BFO)
- the Zachman Framework (Zachman Ontology)

Application

- Ontologies serve as explicit domain models, carrying structure, meaning and reusable information. They can be queried to answer business questions. They enable zooming in/out of complexity and can unify different enterprise perspectives.
- Ontologies support the explicit capture of business rules, aiding to overcome challenges that arise as a result of the hard-coding of these rules in applications. This helps accelerate discovery phases and impact assessments, among others.
- Ontologies are a great way of defining the blueprint for enterprise knowledge graphs. These are extremely powerful assets for unifying enterprise perspectives and bringing machine-interpretable meaning to all sorts of enterprise data.
- Ontologies are an integral part of Linked Open Data (LOD) initiatives. They provide the foundation for describing information structures for remarkably extensive datasets. DBpedia is a great example of linked data using the power of ontologies and knowledge graphs.

Diagram: unified modeling language (UML)



<https://app.diagrams.net/?src=about>

Protege

The screenshot displays the Protege ontology editor interface. At the top is a menu bar with options: File, Edit, View, Reasoner, Tools, Refactor, Window, and Help. Below the menu is a browser-like address bar showing the current ontology's IRI: `untitled-ontology-20` (<http://www.semanticweb.org/elham.nourghassemi/ontologies/2024/4/untitled-ontology-20>). A tab bar below the address bar lists various views: Active ontology, Entities, Individuals by class, OWLViz, DL Query, Individual Hierarchy Tab, OntoGraf, and SPARQL Query. The main workspace is divided into three panes. The top-left pane, titled 'Ontology header', contains the 'Ontology IRI' and 'Ontology Version IRI'. The bottom-left pane, titled 'Annotations', is currently empty. The right pane, titled 'Ontology metrics', displays a table of metrics for the current ontology. The table is organized into three sections: 'Metrics', 'Class axioms', and 'Object property axioms'. All metrics are currently at zero.

Metrics	
Axiom	0
Logical axiom count	0
Declaration axioms count	0
Class count	0
Object property count	0
Data property count	0
Individual count	0
Annotation Property count	0

Class axioms	
SubClassOf	0
EquivalentClasses	0
DisjointClasses	0
GCI count	0
Hidden GCI Count	0

Object property axioms	
SubObjectPropertyOf	0
EquivalentObjectProperties	0

At the bottom of the interface, there is a tab bar with 'Ontology imports', 'Ontology Prefixes', and 'General class axioms'. The 'Ontology imports' tab is active, showing an 'Imported ontologies' section with 'Direct Imports' and 'Indirect Imports' subsections, both of which are currently empty.

Open empty project

Give the IRI

The screenshot shows the OntoGraf interface for the BallpointPenOntology. The main window displays the ontology header with the following information:

- Ontology IRI:** <http://example.com/BallpointPenOntology>
- Ontology Version IRI:** <http://example.com/BallpointPenOntology/1.0.0>

The right sidebar shows the ontology metrics:

Metrics	
Axiom	0
Logical axiom count	0
Declaration axioms count	0
Class count	0
Object property count	0
Data property count	0
Individual count	0
Annotation Property count	0

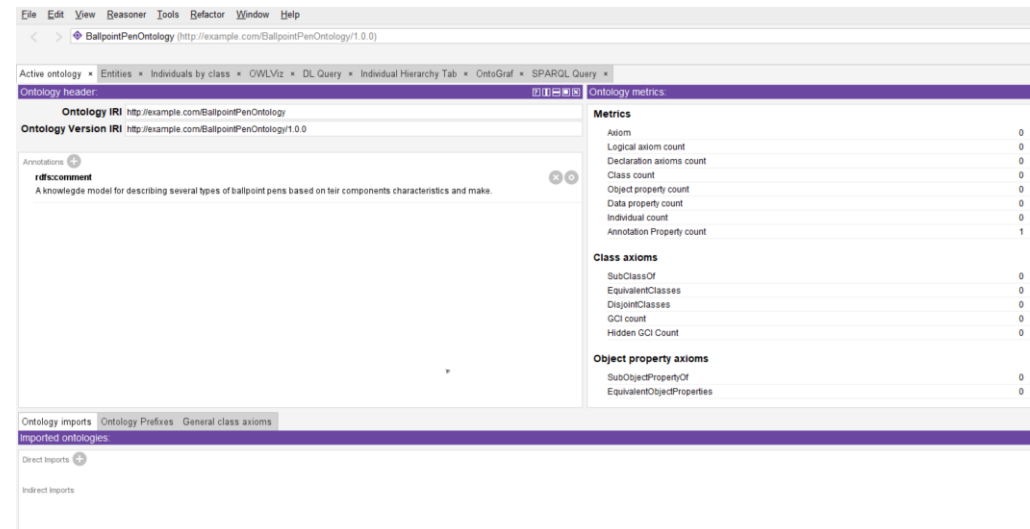
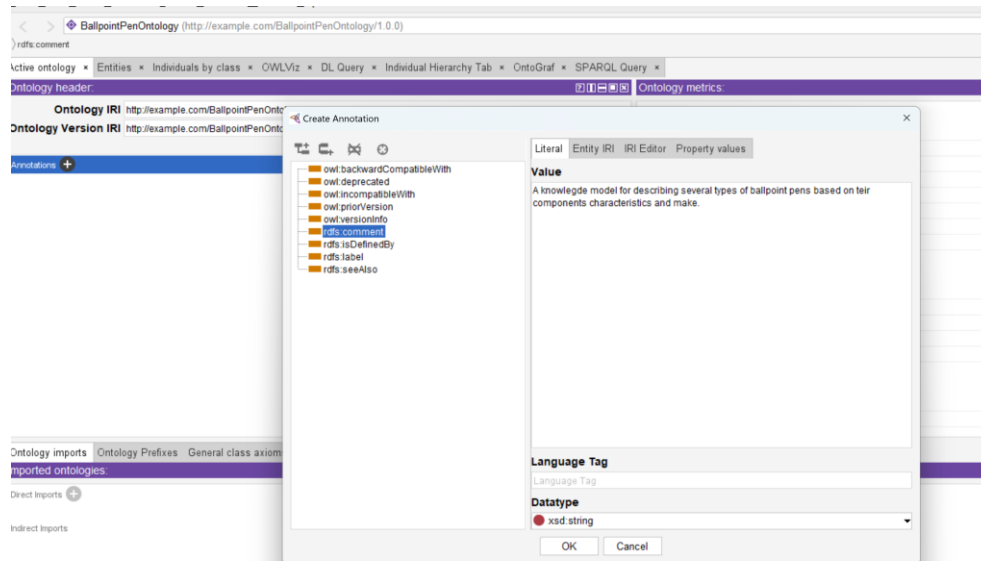
The bottom section shows the ontology imports:

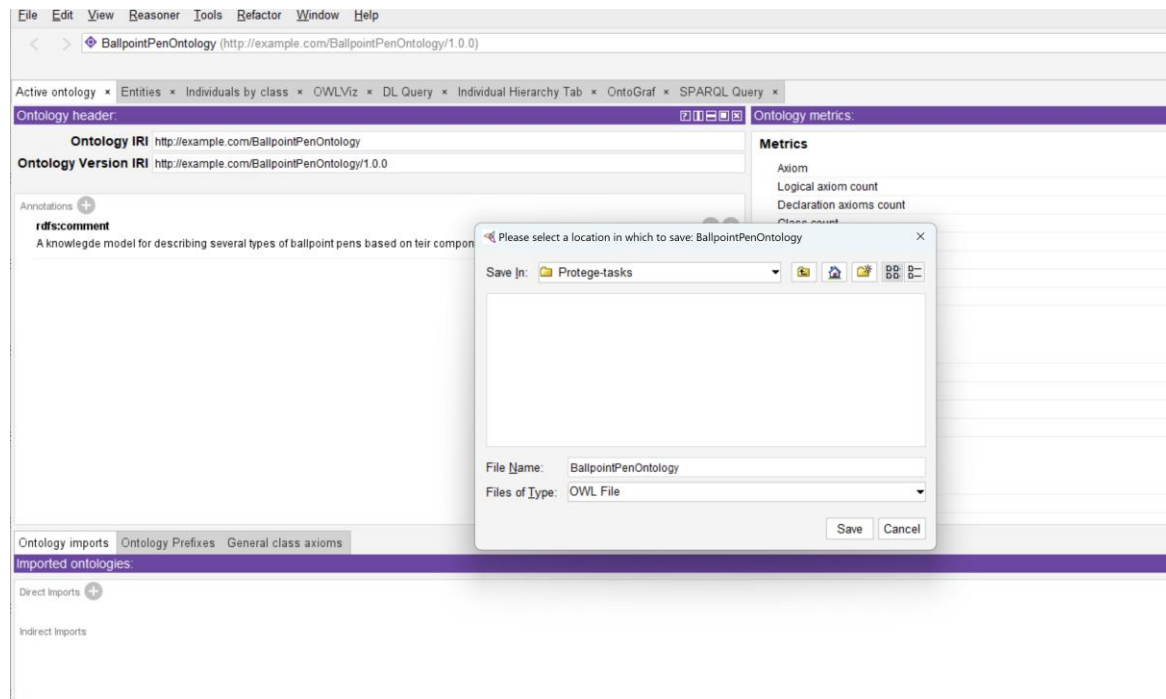
Class axioms	
SubClassOf	0
EquivalentClasses	0
DisjointClasses	0
GCI count	0
Hidden GCI Count	0

Object property axioms	
SubObjectPropertyOf	0
EquivalentObjectProperties	0

IRI is a unique identifier for ontology which we call the namespace.

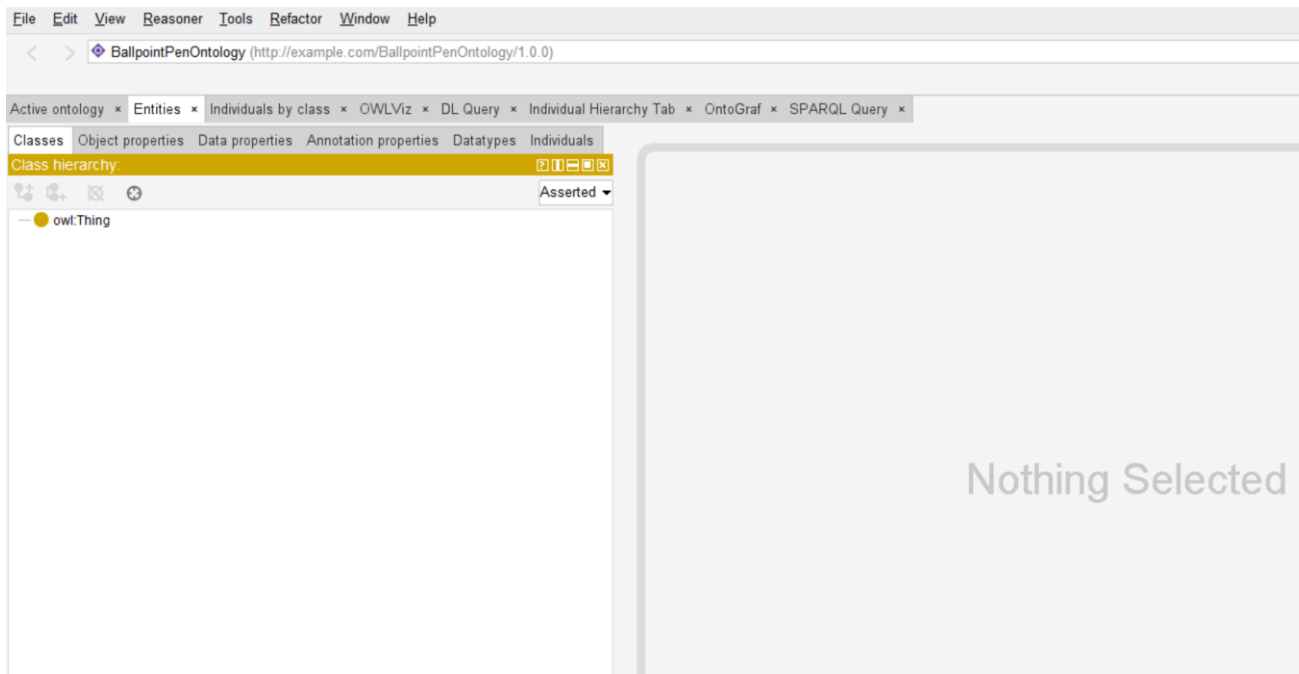
- Add a comment to ontology as description of what it is.





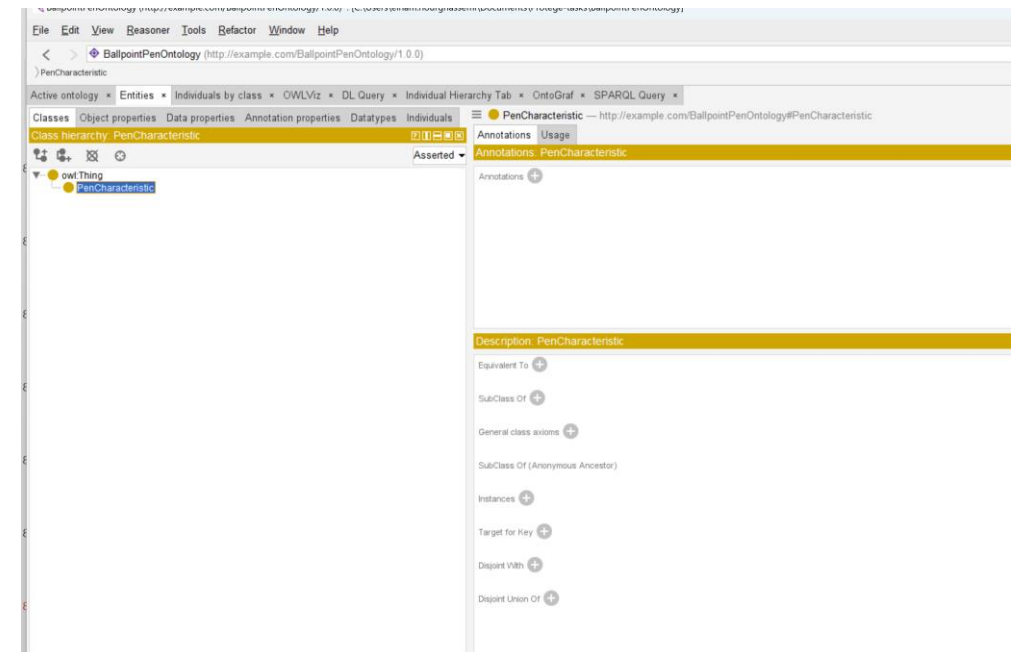
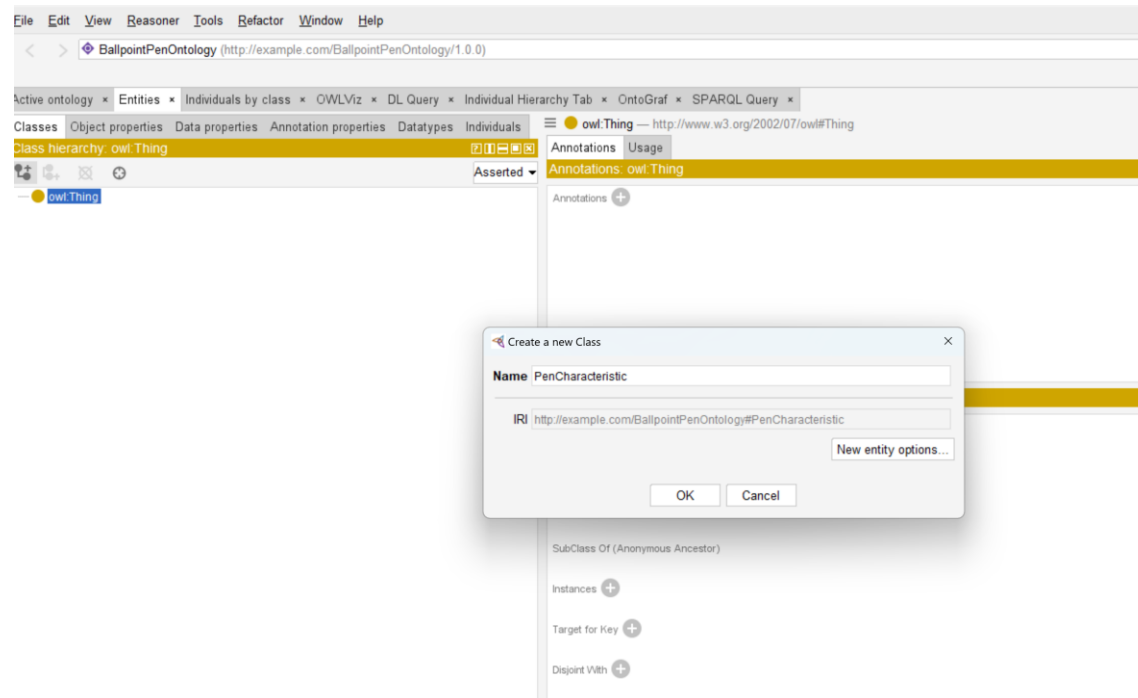
Save as default
format as the
RDF/XML

- Classes and their hierarchy constitute the backbone of OWL ontologies.
- Switch to classes tab which is available under the entities tab

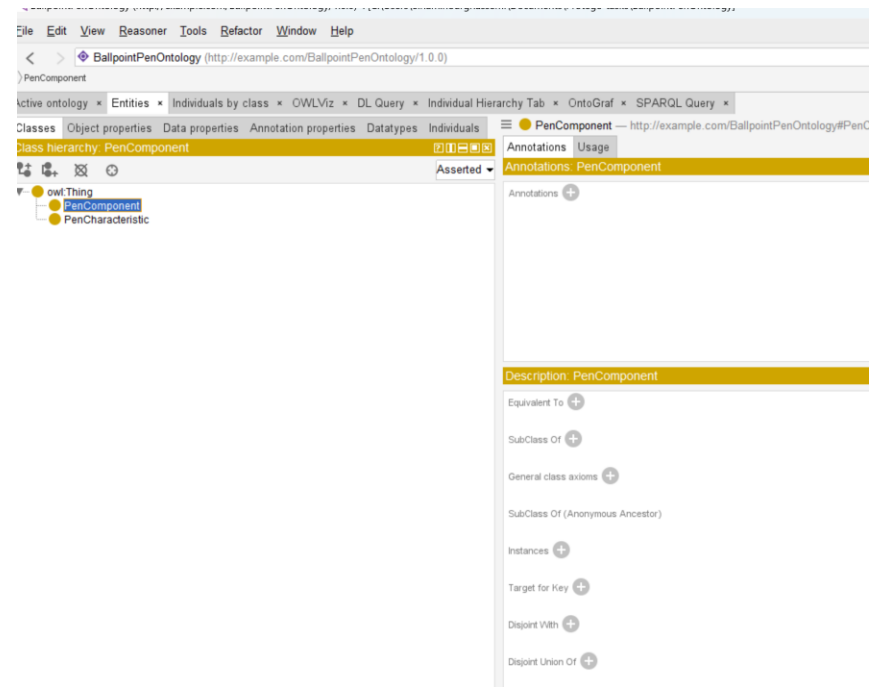
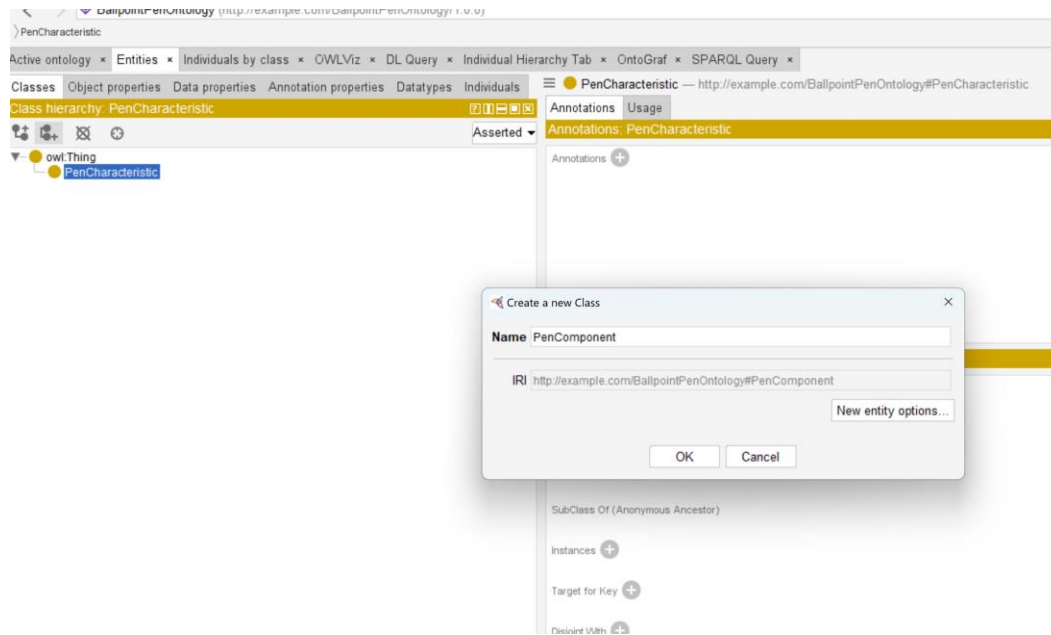


An empty OWL ontology contains one class called owl: Thing which is a root class and this is by design. So basically all the classes we create are implicitly subclasses of owl:Thing

- Add classes



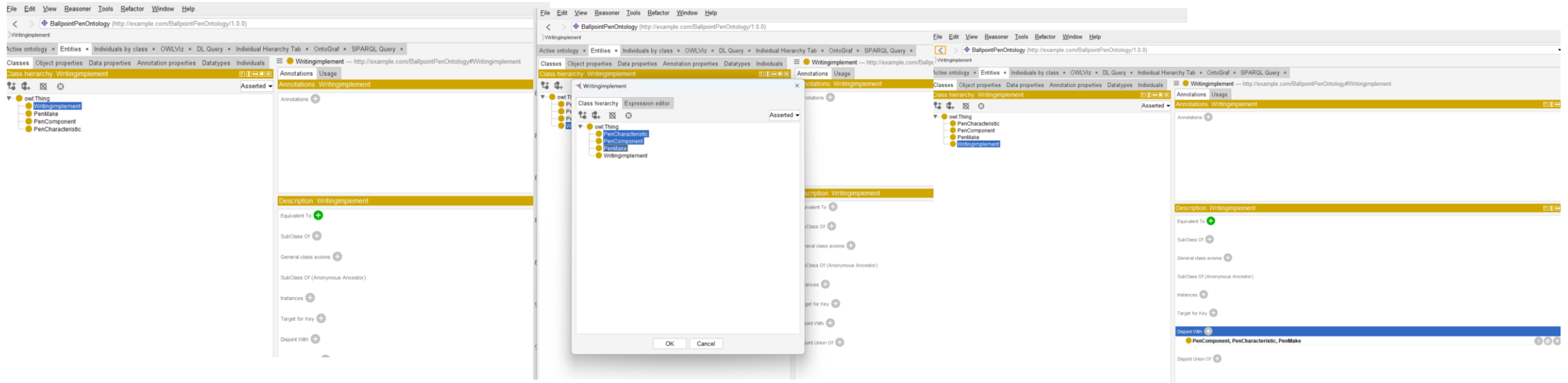
- Add sibling class



Disjoint classes

They are disjoint, that the individuals of these classes are all distinct kinds of things and there is no cross-over, which means that an individual of PenCharacteristic cannot be an individual of PenComponent or PenMake,....

Let's specify the classes we created are disjoint.



Add some subclasses

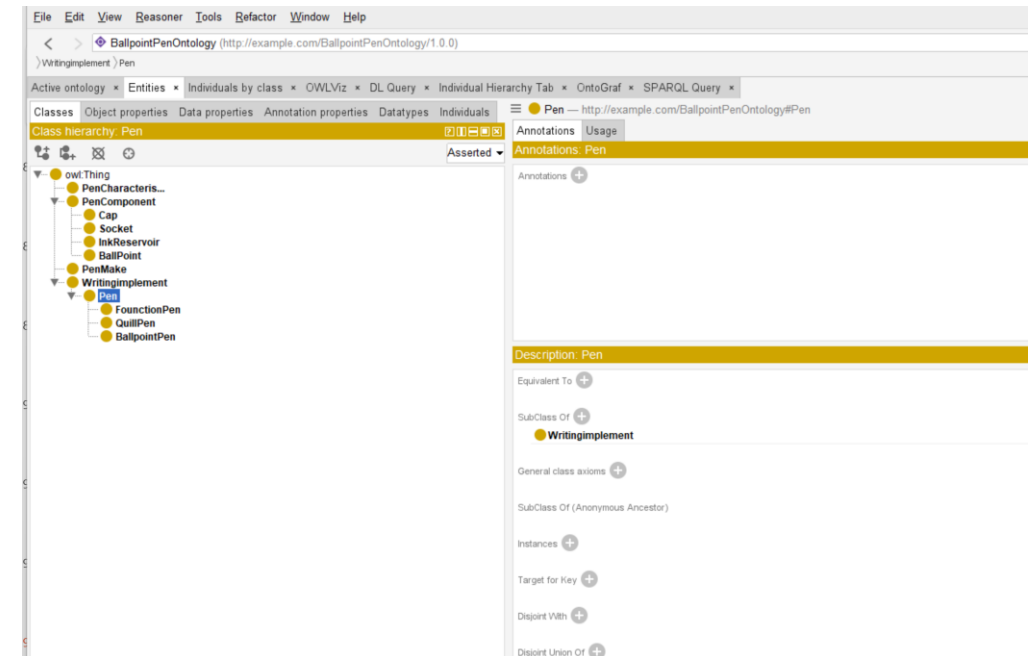
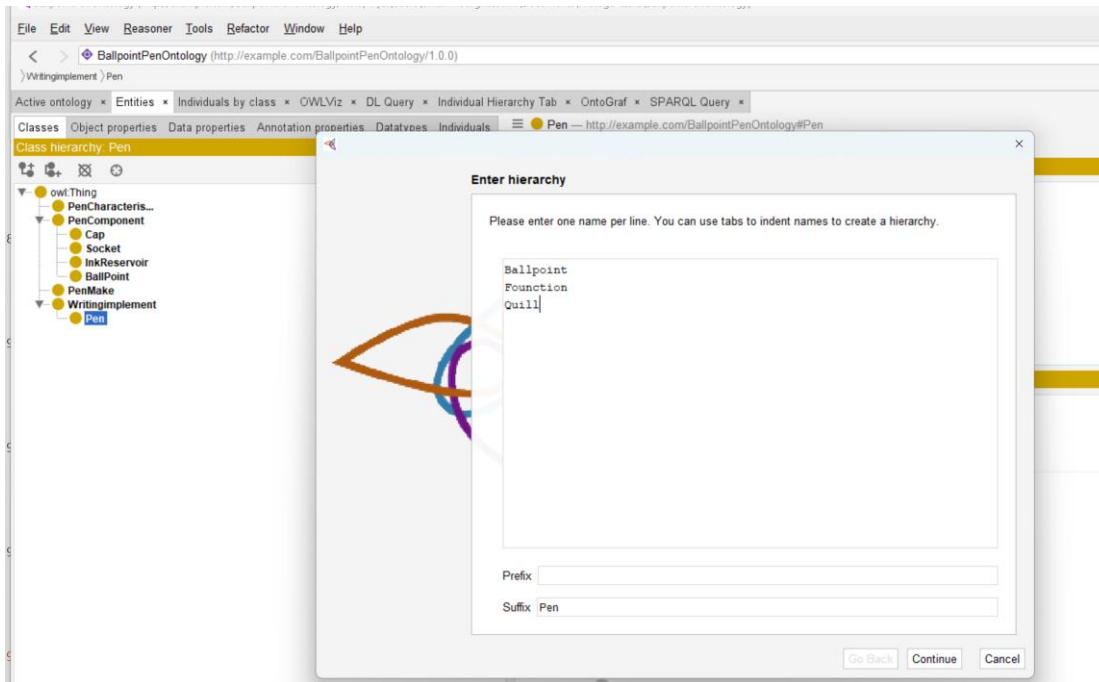
The screenshot shows the Protege ontology editor interface. The top menu bar includes File, Edit, View, Reasoner, Tools, Refactor, Window, and Help. The address bar displays the ontology URI: <http://example.com/BallpointPenOntology/1.0.0>. The main workspace is titled "Class hierarchy: Pen" and shows a tree view of the ontology classes. The tree structure is as follows:

- owl:Thing
 - PenCharacteris...
 - PenComponent
 - Cap
 - Socket
 - InkReservoir
 - BallPoint
 - PenMake
 - Writingimplement
 - Pen

The right-hand pane shows the "Annotations: Pen" tab, which is currently empty. Below this, the "Description: Pen" tab is active, displaying a list of class axioms and relationships, including "Equivalent To", "SubClass Of", "General class axioms", "SubClass Of (Anonymous Ancestor)", "Instances", "Target for Key", and "Disjoint With". The "SubClass Of" relationship is highlighted, showing that "Pen" is a subclass of "Writingimplement".

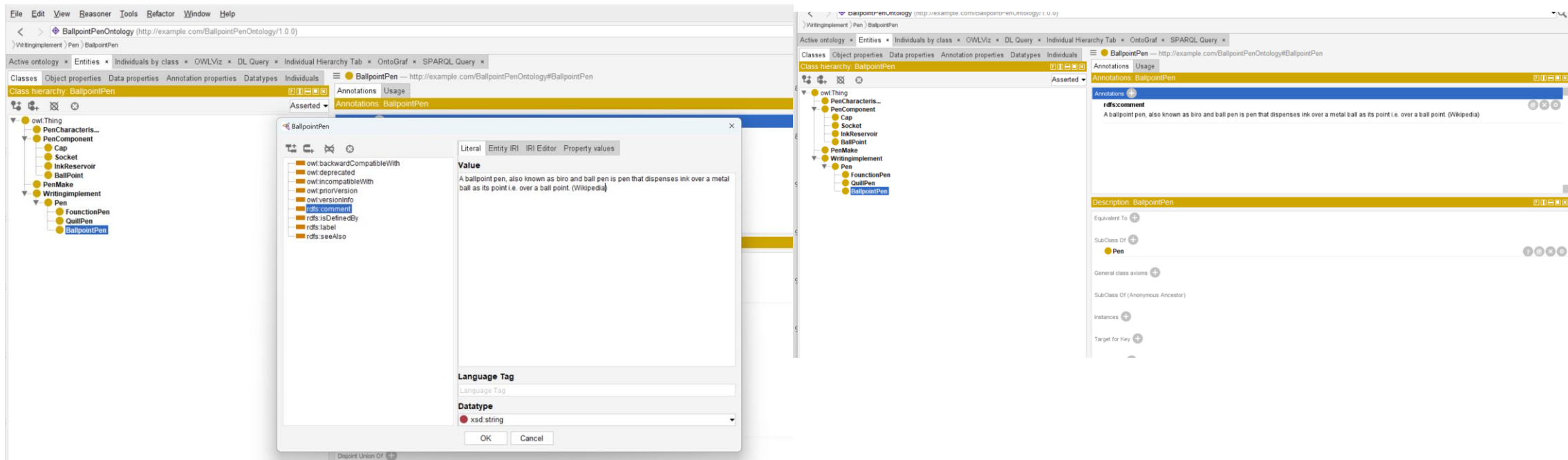
Class Hierarchy

- From the Tools menu select the option Create Class Hierarchy
- Our root class is already selected



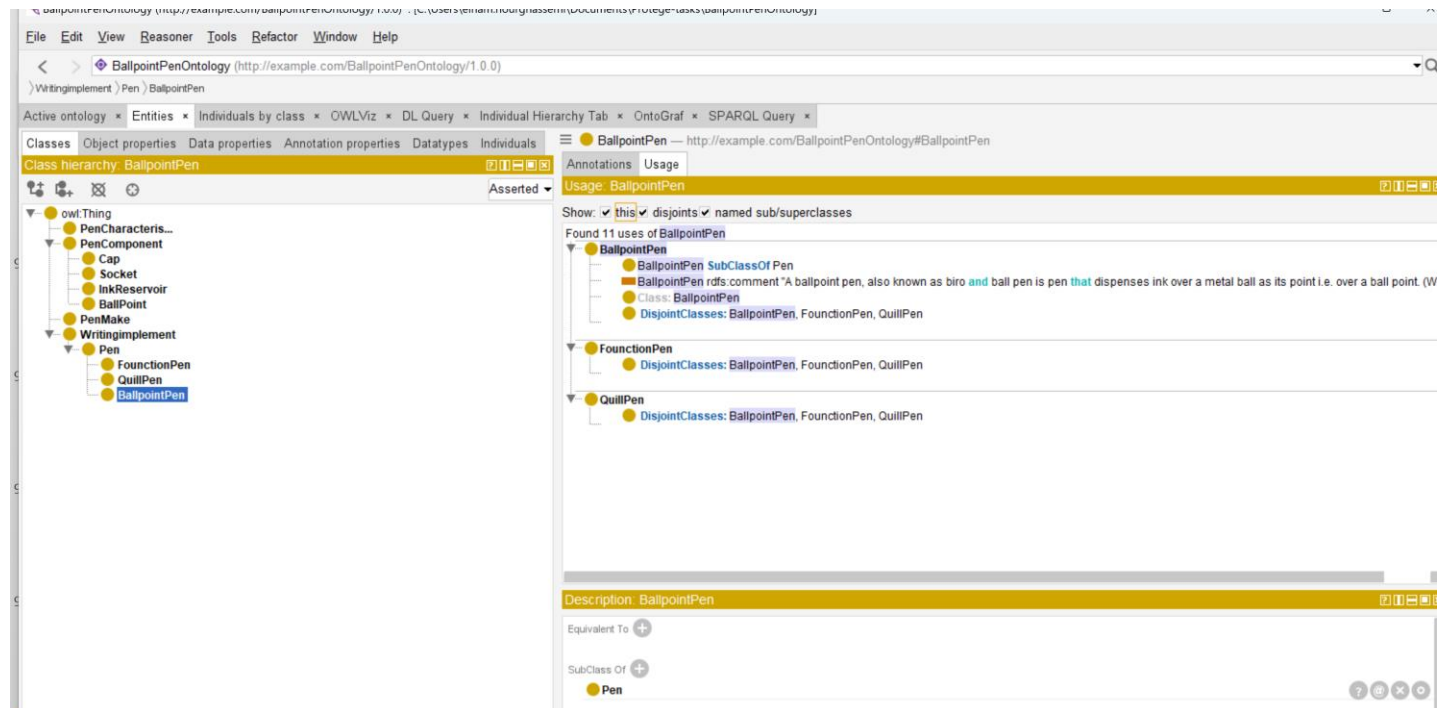
Add comment

- Use the Annotations section in the class view to add a comment and type the description



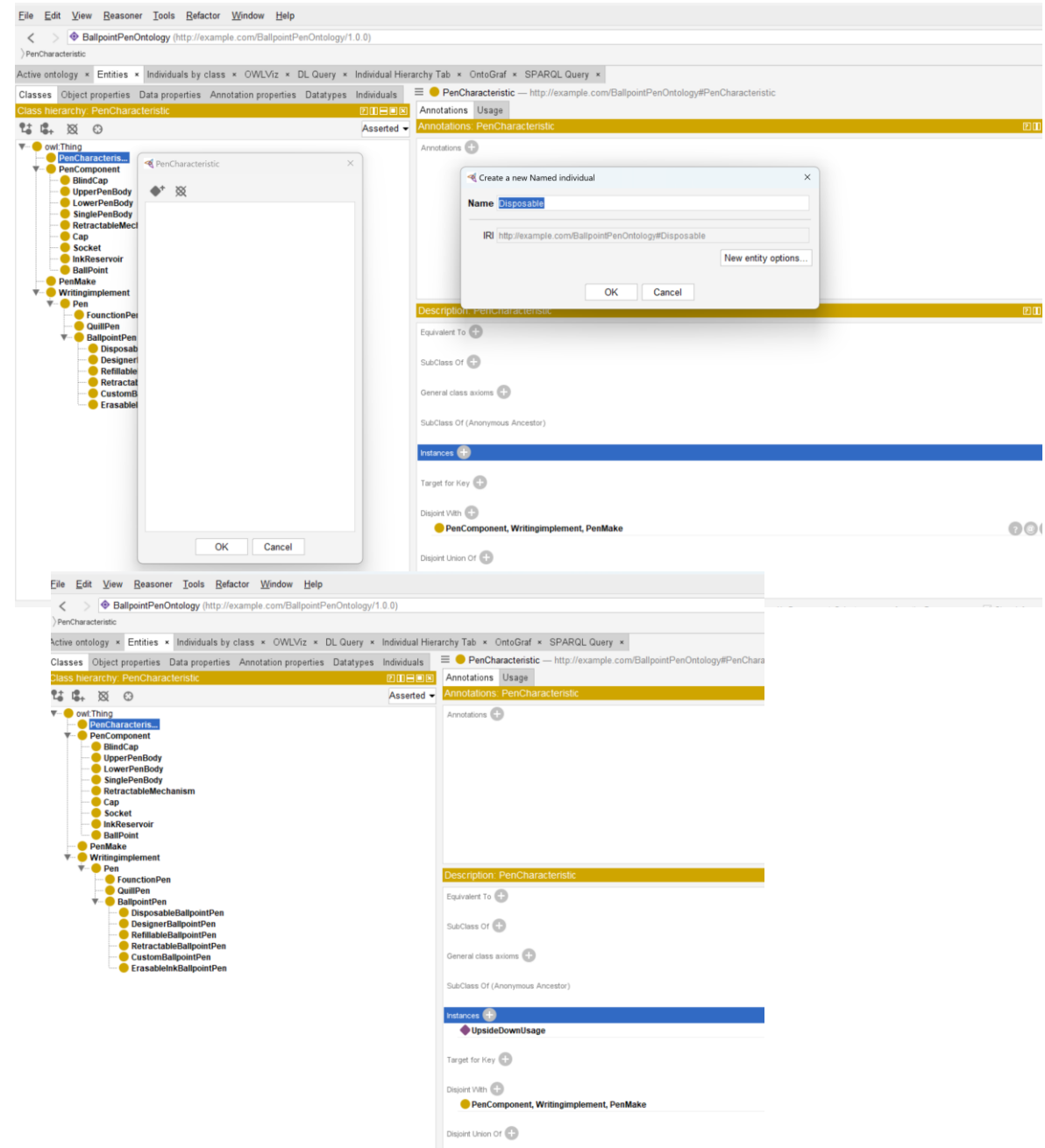
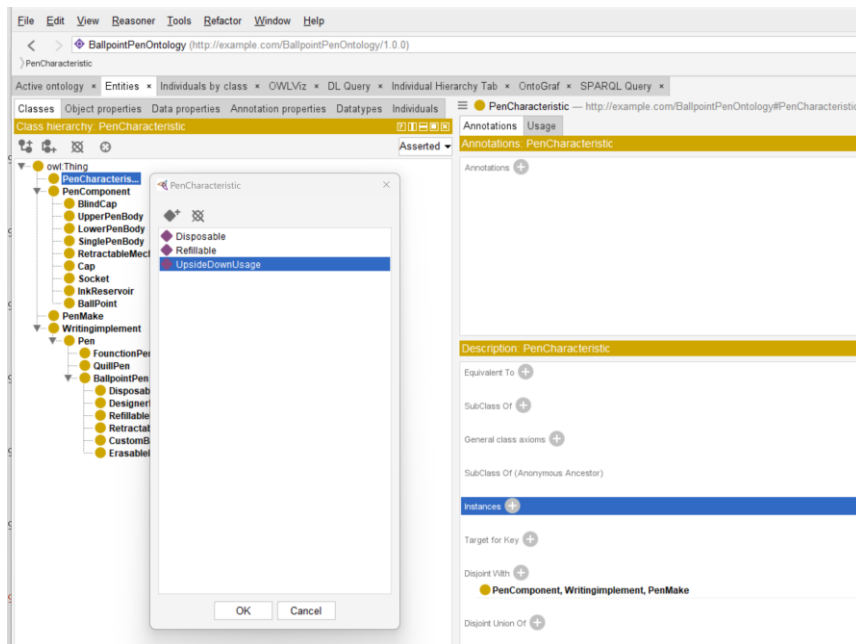
Usage tab

- By switching to this tab we are able to see where in our ontology class is being used.

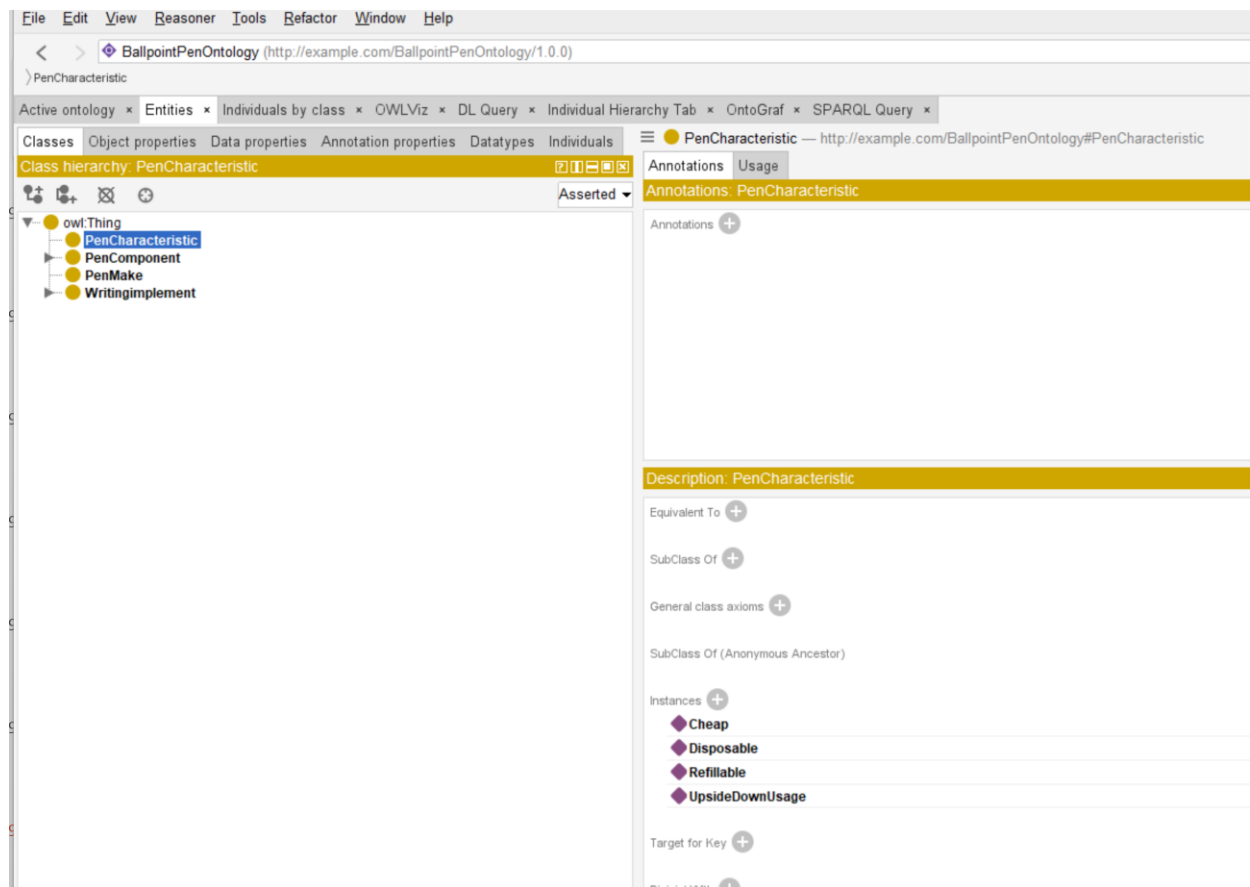
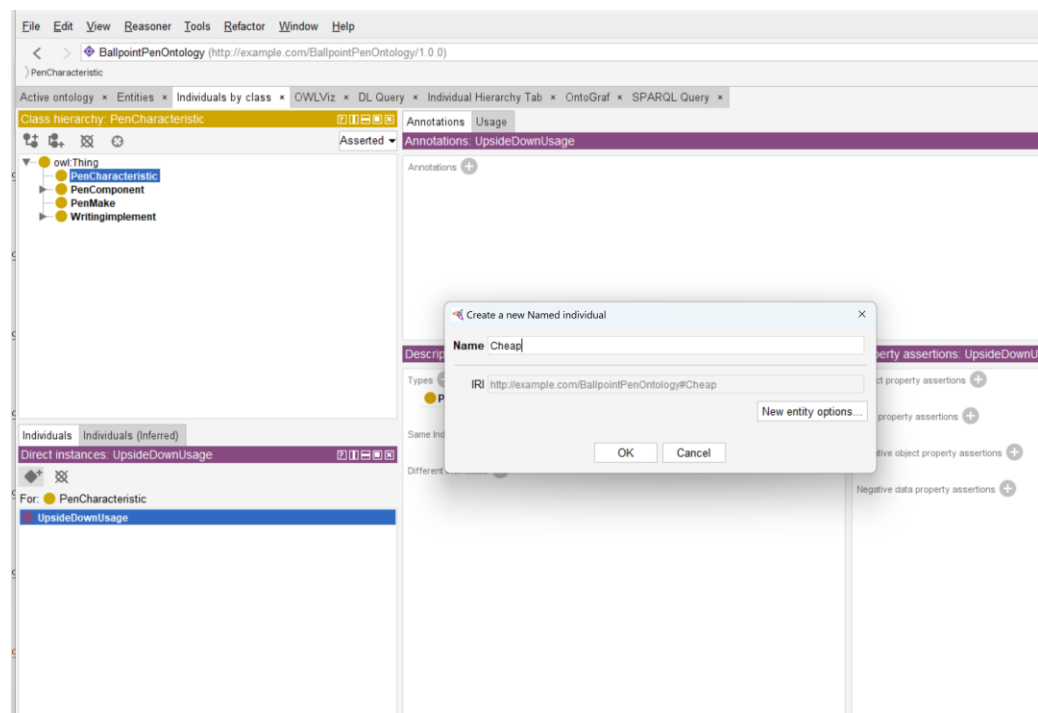


Owl individuals

- Add instances

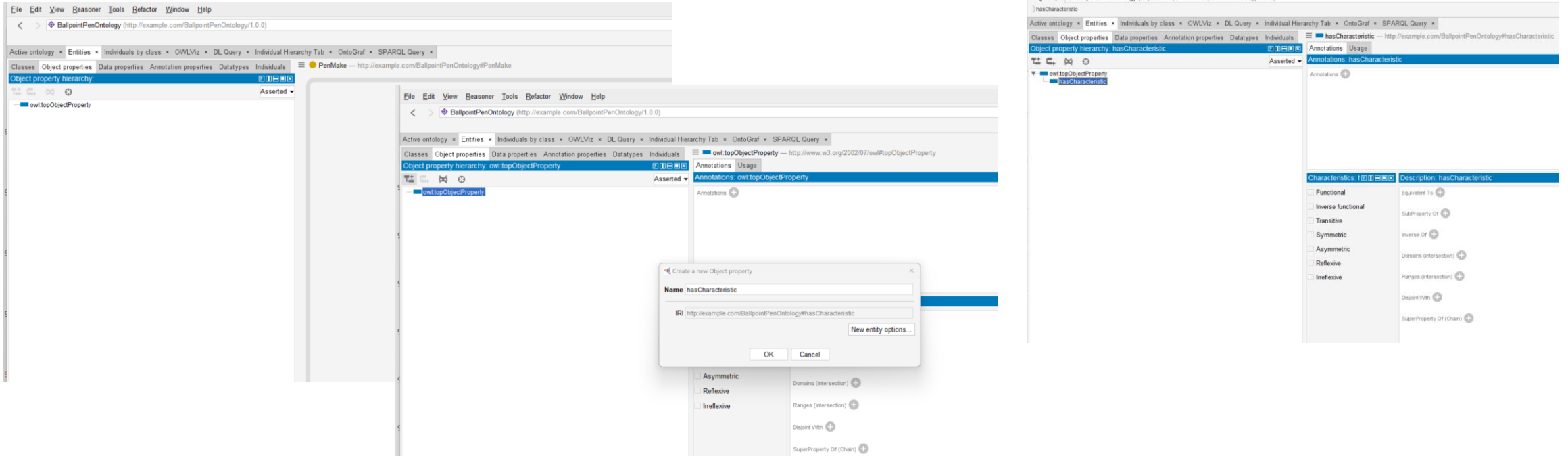


The other way to use individuals by class tab

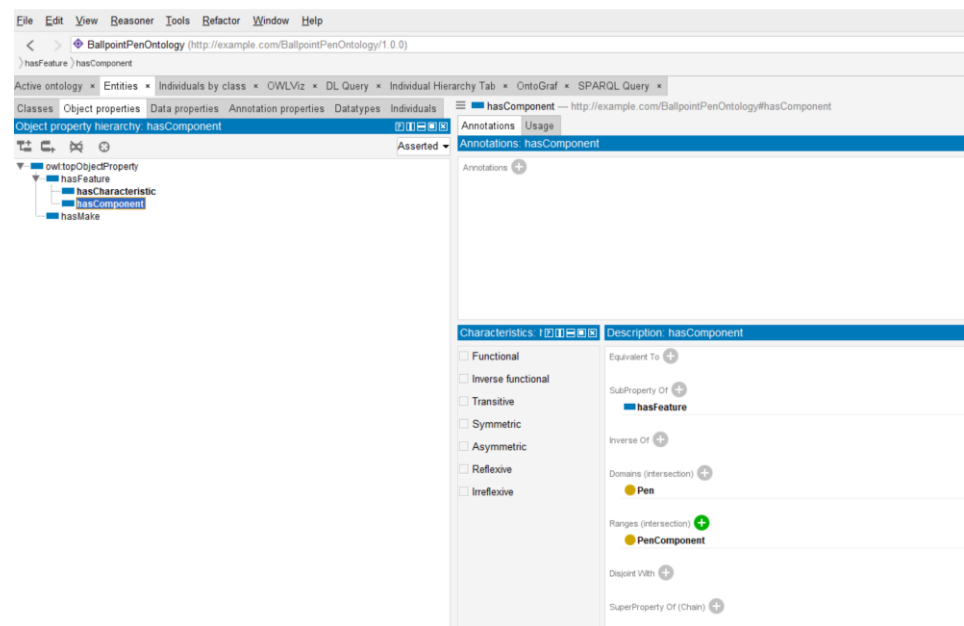
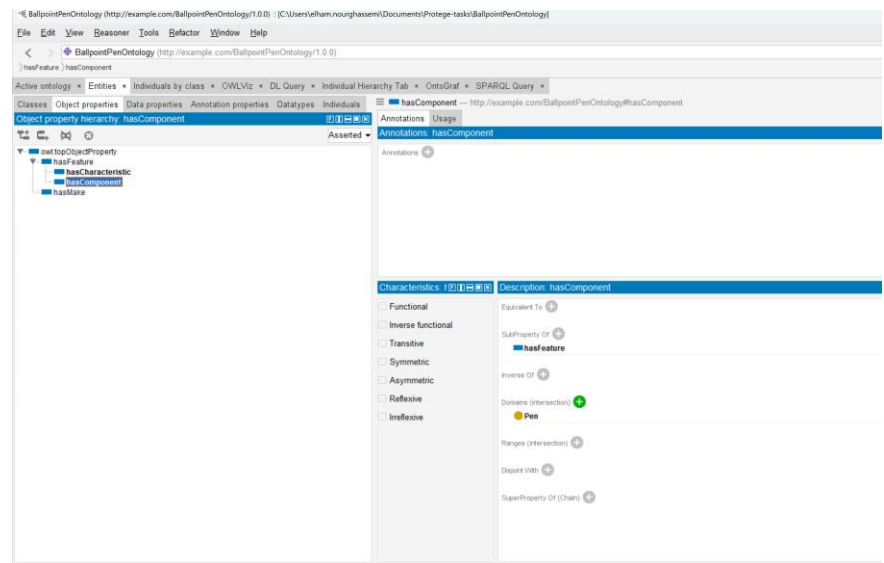
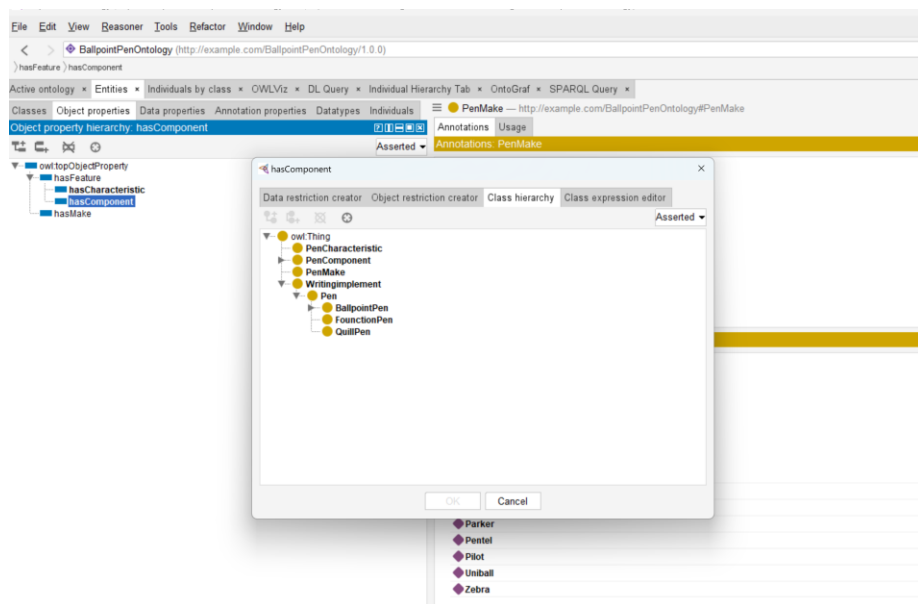


OWL properties

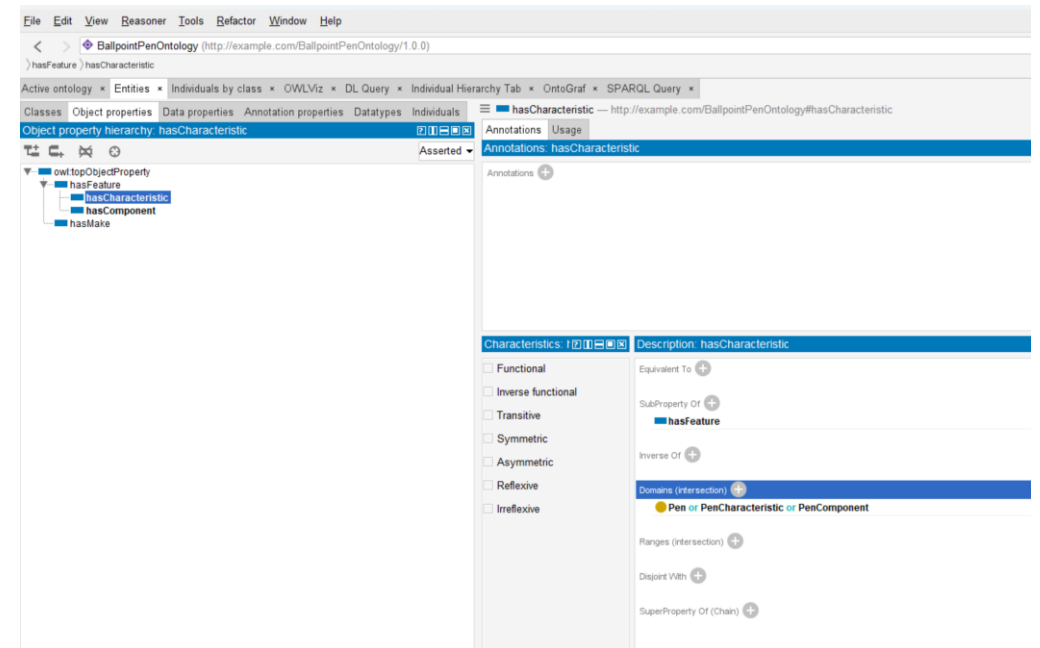
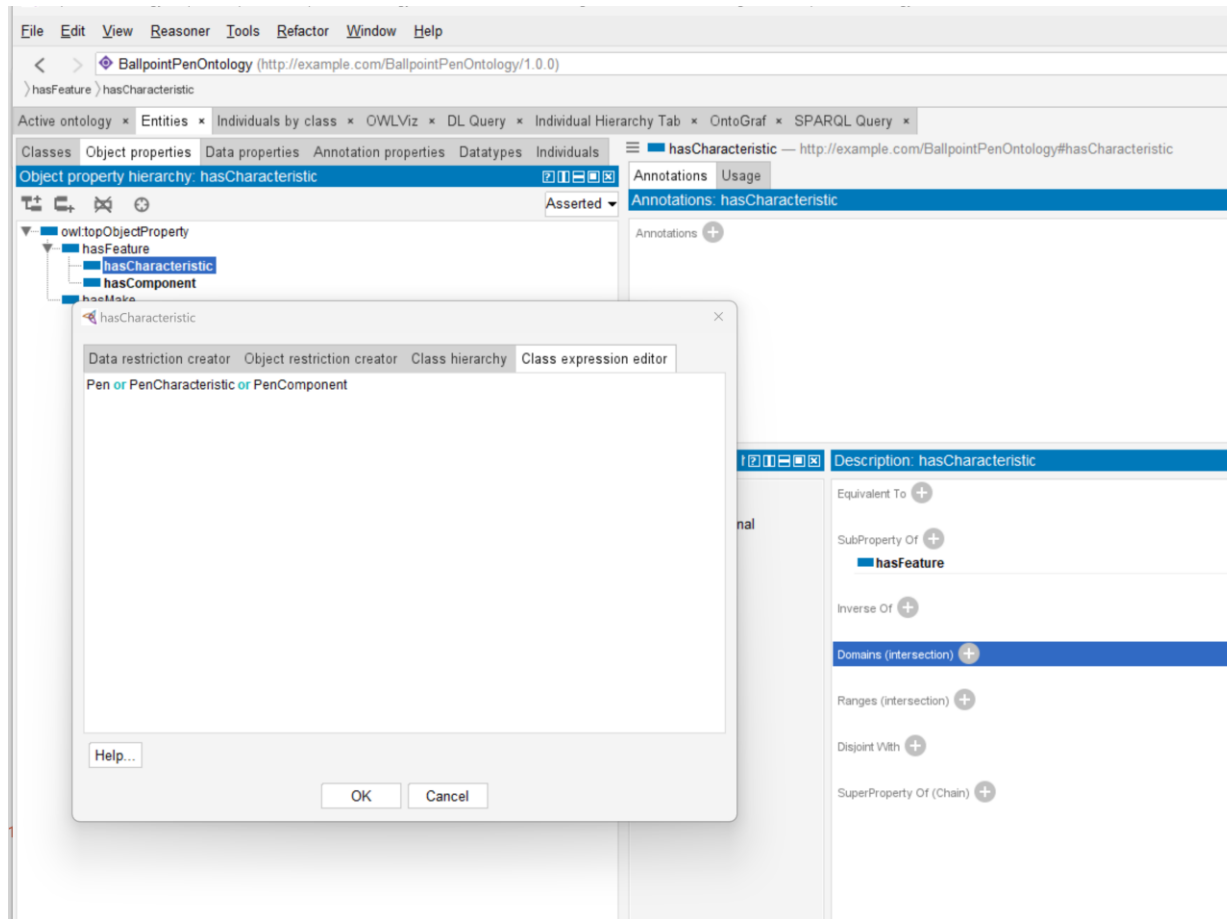
- Switch to the object properties tab



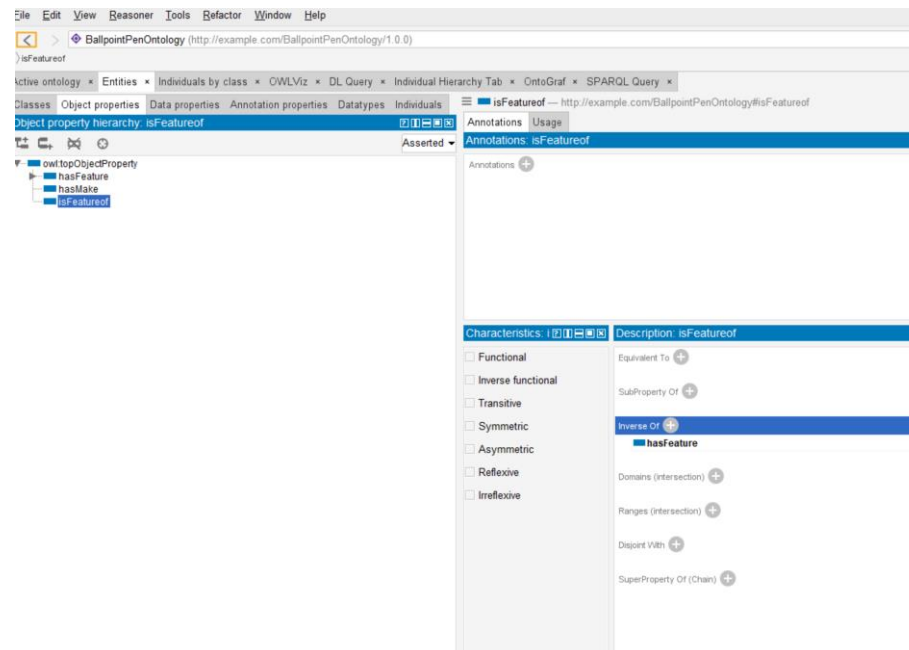
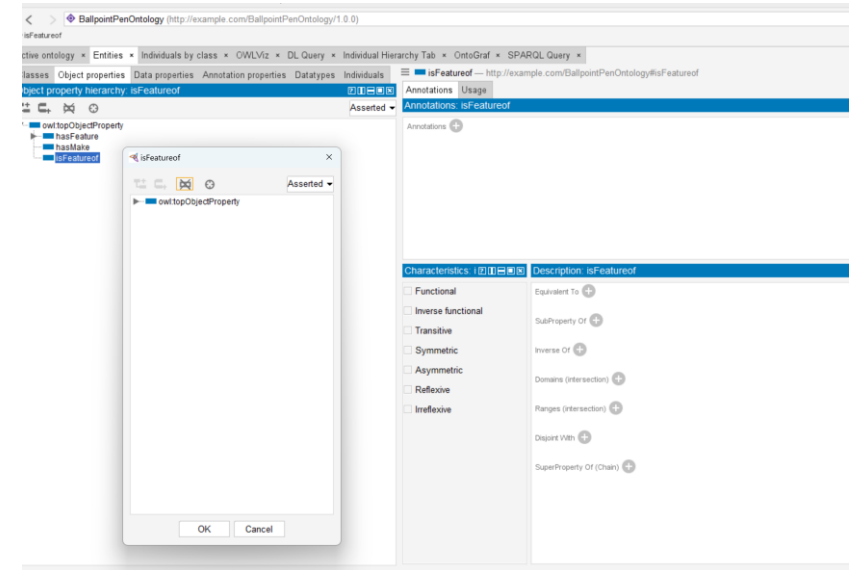
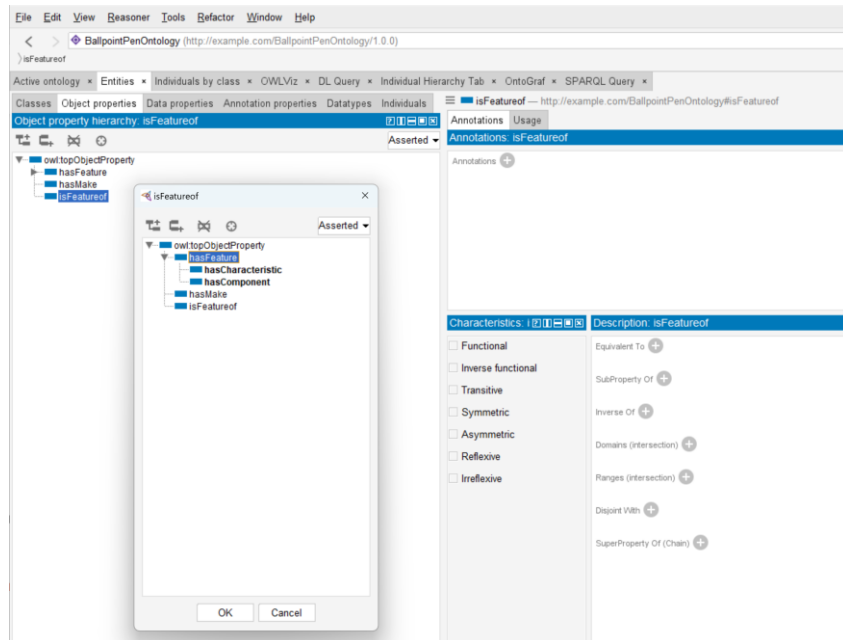
Add domain and range



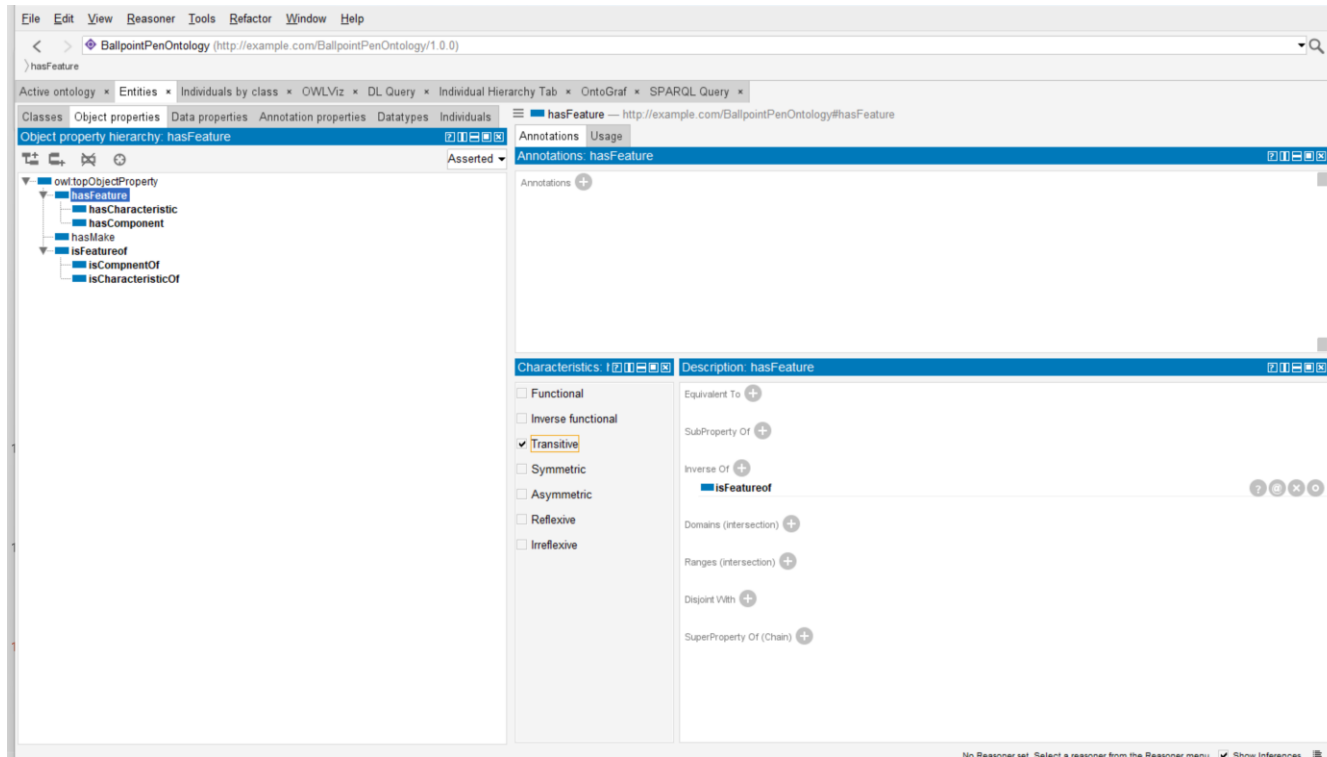
Union of the classes domain



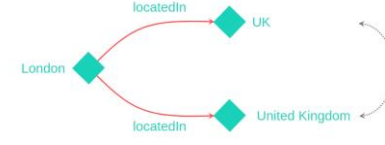
Inverse properties



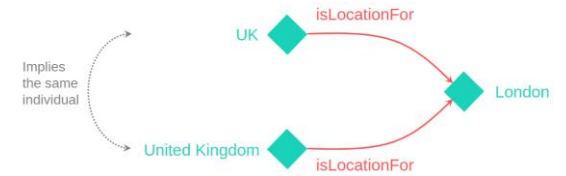
Characteristics



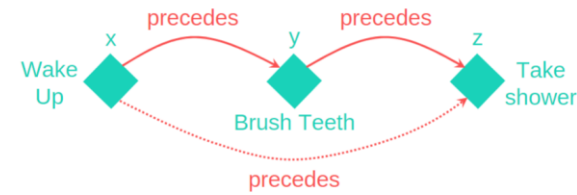
Functional Properties



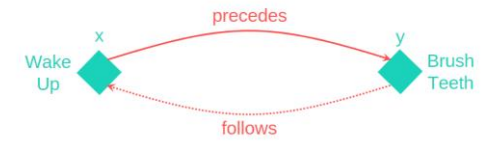
Inverse Functional Properties



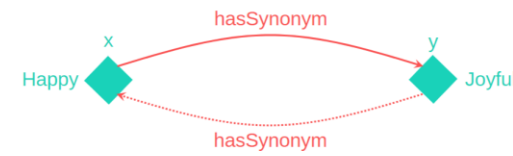
Transitive Properties



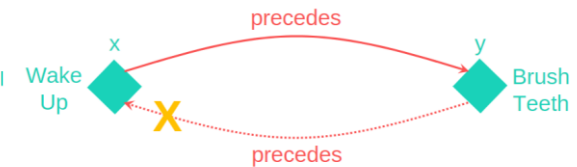
Inverse Properties



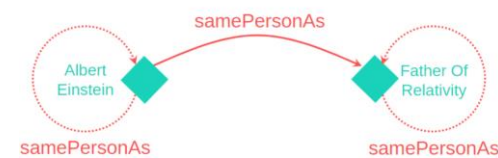
Symmetric Properties



Asymmetric Properties



Reflexive Properties



Irreflexive Properties



OWL restrictions

OWL Restrictions



Quantifier
Restriction

“hasValue”
Restriction

Cardinality
Restriction

- Existential restriction
- Universal restriction

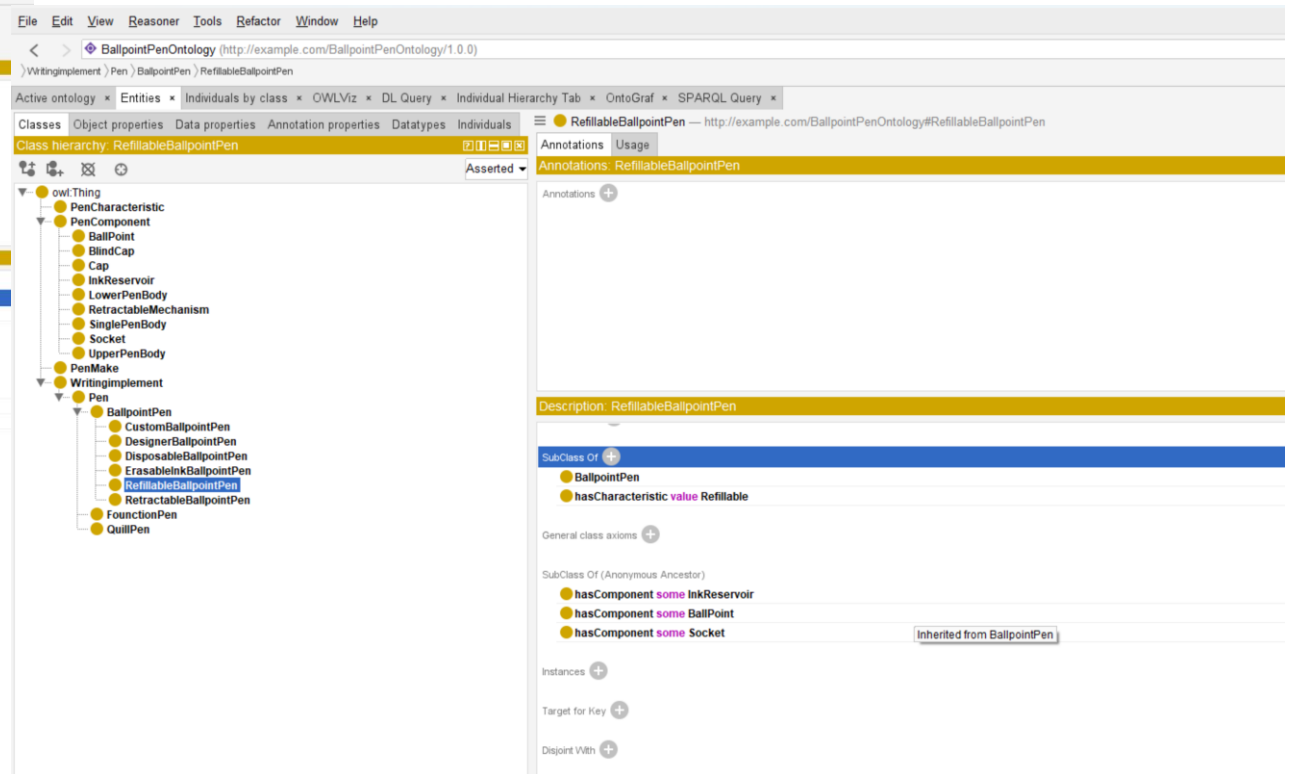
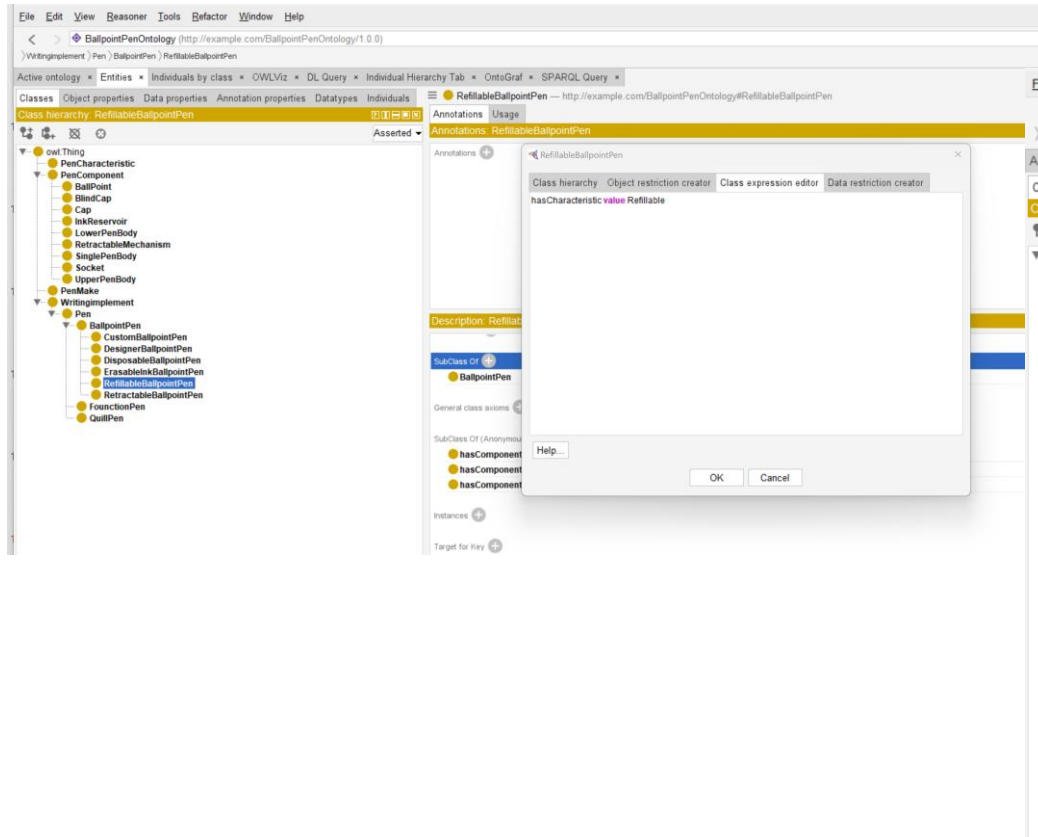
Add restriction

The image displays two screenshots from the Protégé ontology editor, illustrating the process of adding a restriction to an ontology.

Left Screenshot: The main window shows the "PenMake" class hierarchy. The "PenMake" class is selected. A dialog box titled "PenMake" is open, showing the "Restricted property" and "Restriction filter" tabs. The "Restricted property" tab is active, showing a tree structure with "hasFeature" and "hasComponent" properties. The "Restriction type" is set to "Some (existential)" and the "Cardinality" is set to "1".

Right Screenshot: The main window shows the "BallpointPen" class hierarchy. The "BallpointPen" class is selected. The "Description" tab is active, showing the class description: "A ballpoint pen, also known as biro and ball pen is pen that dispenses ink over a metal ball as its point i.e. over a ball point. (Wikipedia)". The "SubClass Of" tab is also visible, showing the class hierarchy.

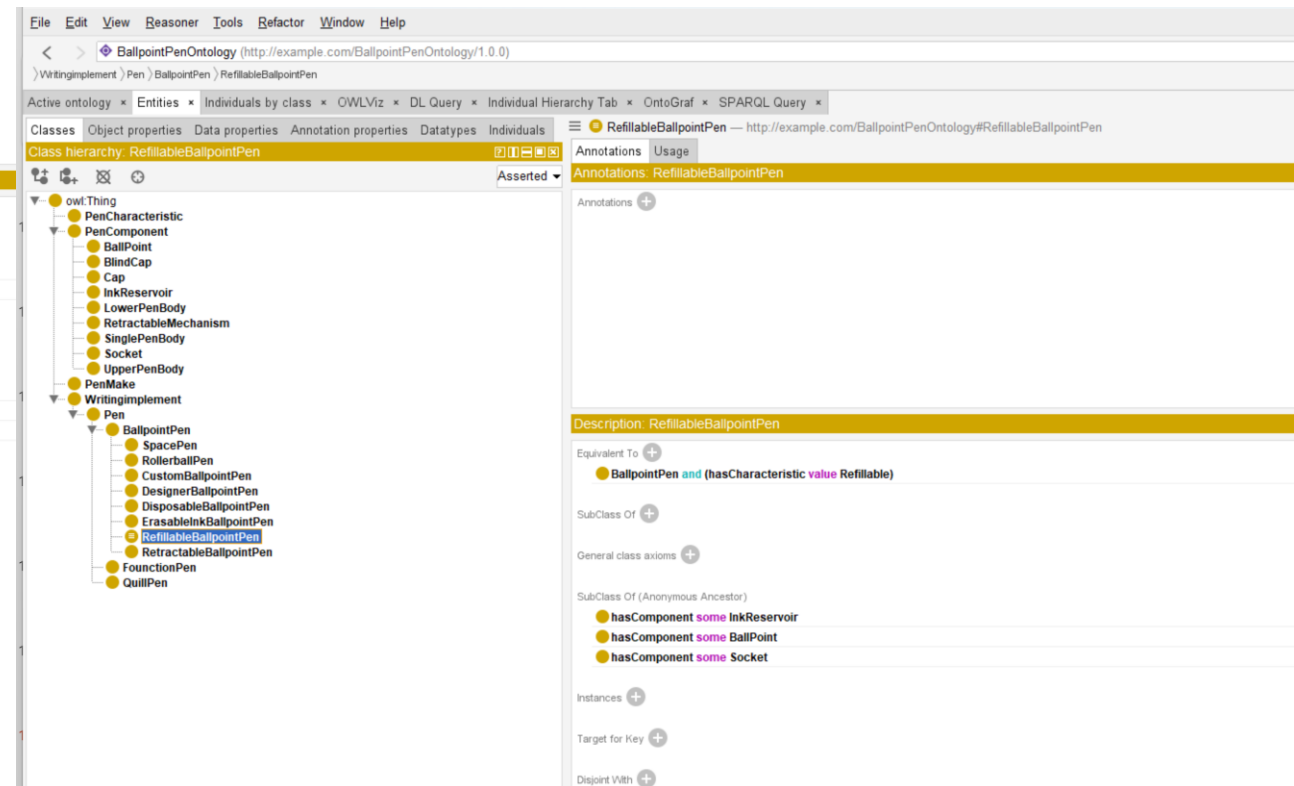
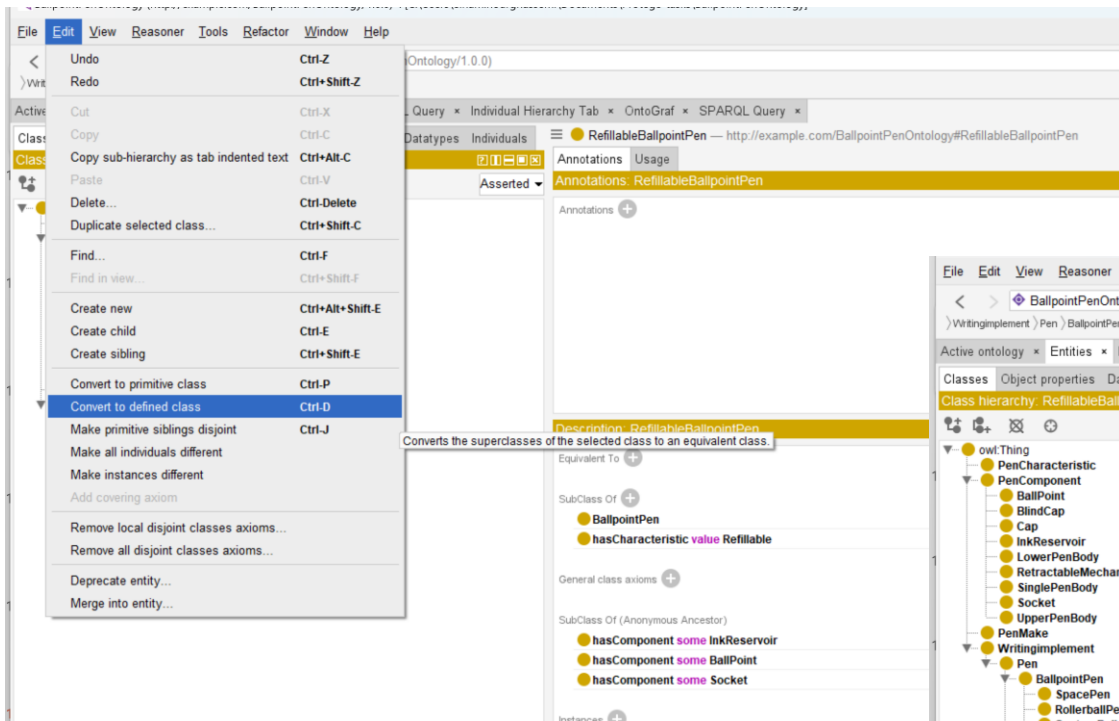
hasValue restrictions



Primitive and defined classes

- In Owl, all classes we describe through restrictions are known as “Primitive classes”. These classes make use of necessary conditions in their description- that is for something to be an individual of a class.
- If we have an individual that satisfies the conditions, the system can infer that the individual in question is a member of our class. In Owl, a class that has a set of necessary and sufficient conditions is known as a “defined class”.

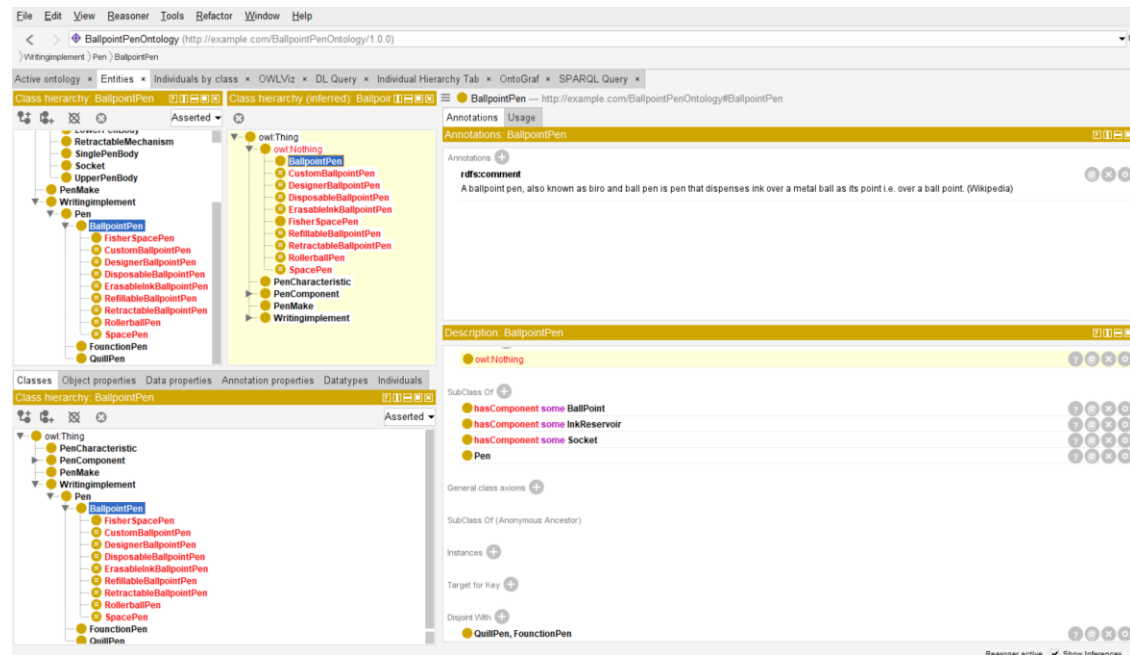
Defined class



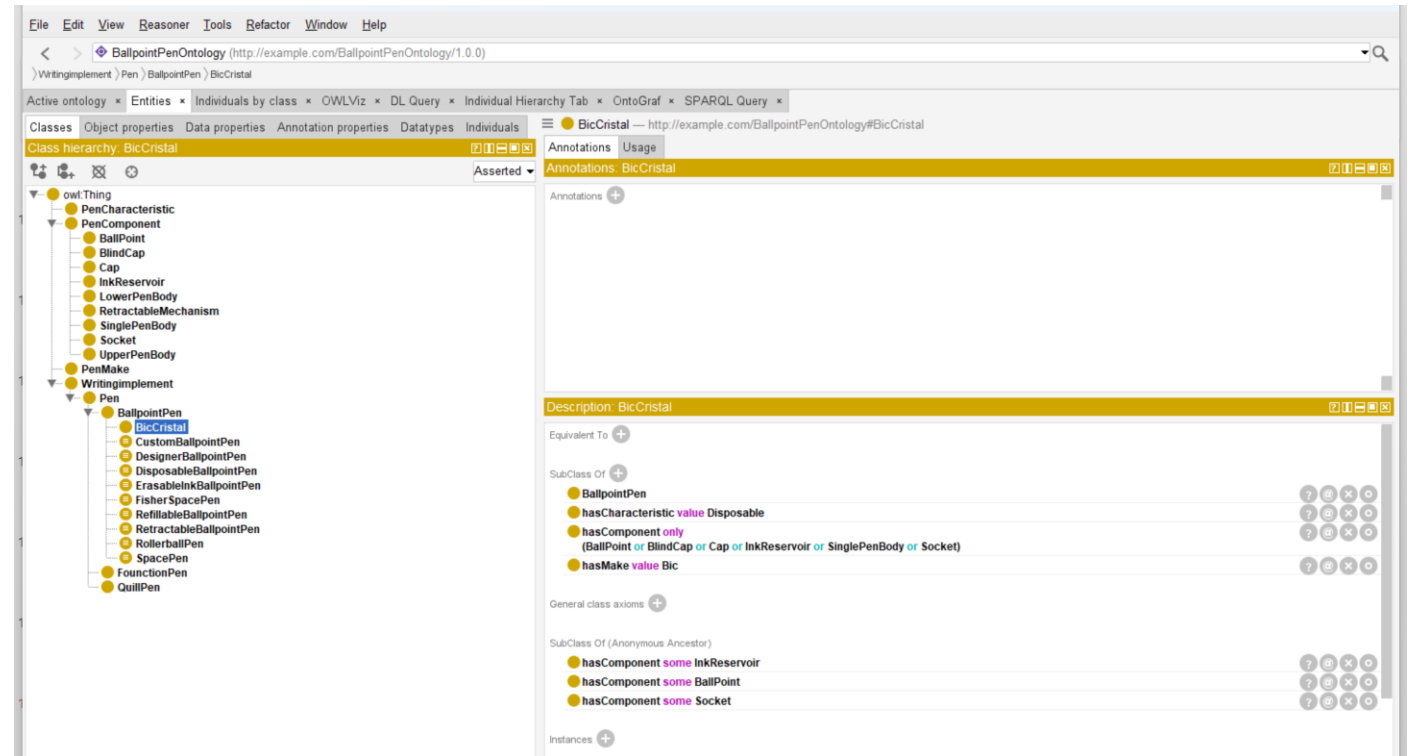
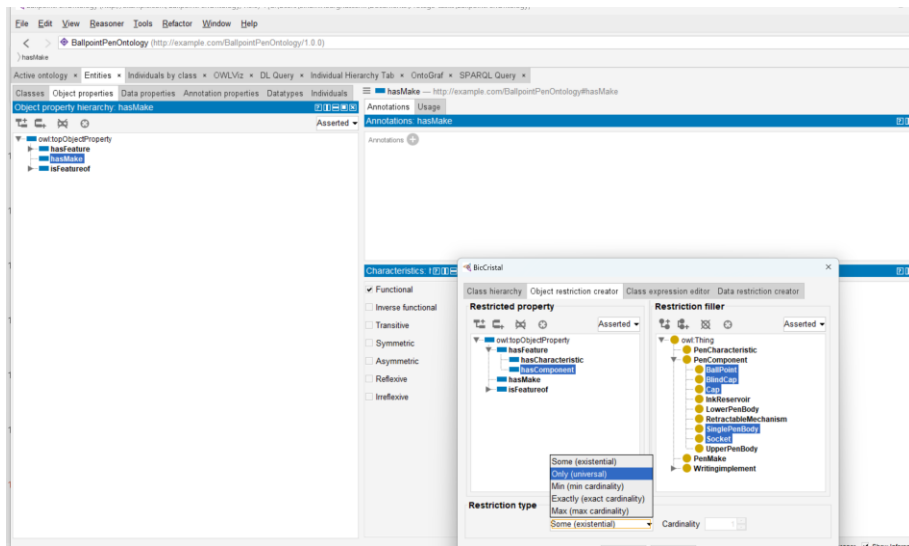
Automated services

Ensure the Show Inferences checkbox is ticked (this is normally at the bottom right-hand corner of the user interface). Then invoke a reasoner by going to Reasoner in the menu bar and selecting, for example, the HermiT option or other listed reasoner (if available)

Add the inferred class hierarchy pane to the Classes tab by going to Window in the menu bar, Views > Class views > Class hierarchy (inferred) and click to add the view on the user interface



Universal restrictions

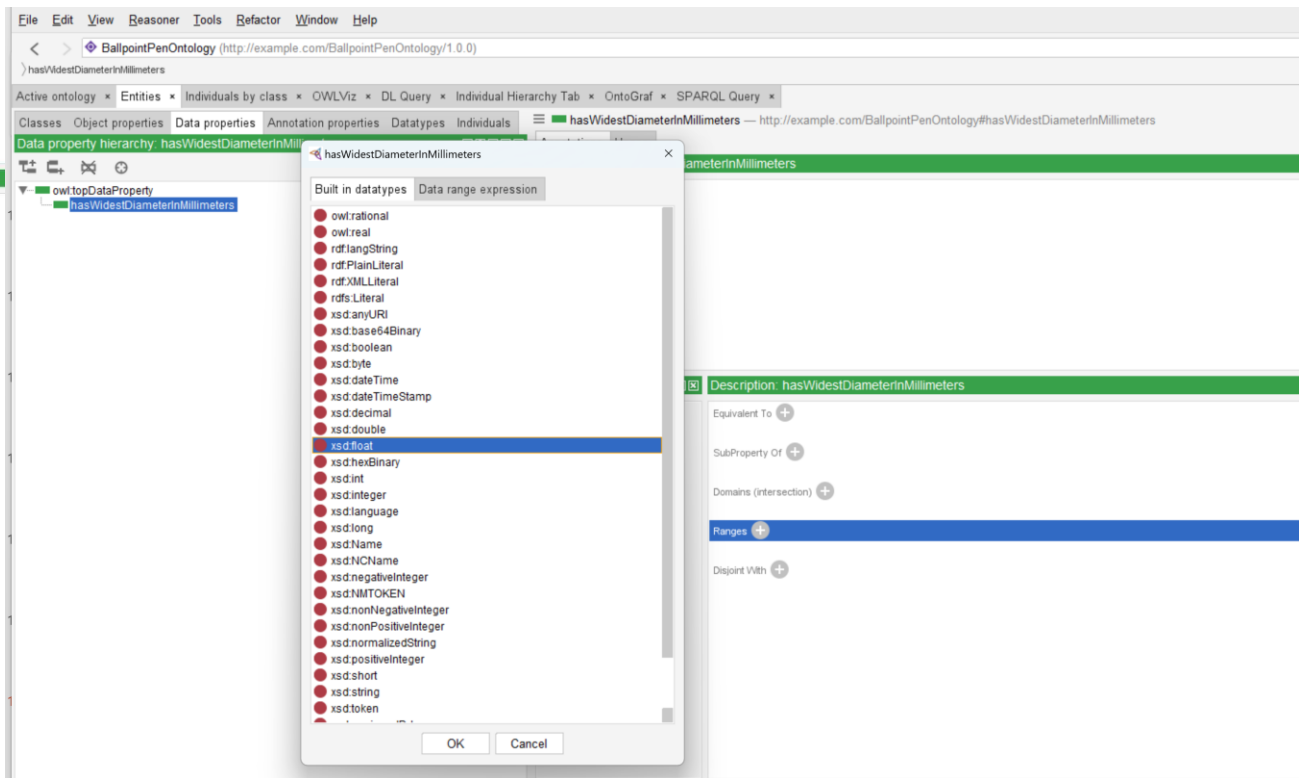
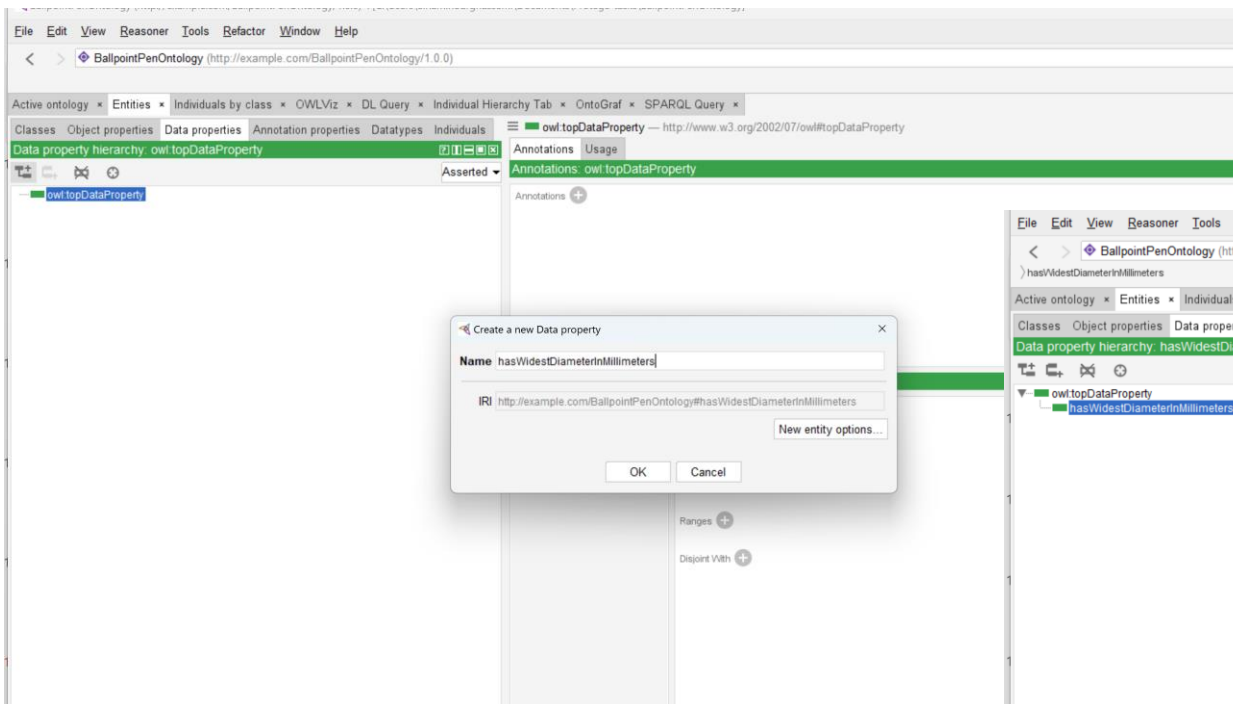


Cardinality restrictions

The image displays two screenshots of the Protege ontology editor, illustrating the process of applying cardinality restrictions to the `MultiColourBallpointPen` class.

Left Screenshot: The `MultiColourBallpointPen` class is selected in the class hierarchy. The `Object restriction creator` dialog is open, showing the `hasComponent` property. The `Restriction type` is set to `Min (min cardinality)`, and the `Cardinality` is set to `2`. The `Restricted property` is `hasComponent`, and the `Restriction filler` is `Socket`.

Right Screenshot: The `MultiColourBallpointPen` class is selected in the class hierarchy. The `Annotations` tab is open, showing the `hasComponent` property with a cardinality restriction of `min 2`. The `MultiColourBallpointPen` class is also shown in the `SubClass Of` list, indicating it is a subclass of `BallpointPen`.



Add instances

The image displays two screenshots of the OntoGraf application, illustrating the steps to add a new instance to an ontology.

Left Screenshot: The application window shows the 'BallpointPenOntology' (http://example.com/BallpointPenOntology/1.0.0). The 'Class hierarchy' tab is active, showing a tree structure with classes like `owl:Thing`, `PenCharacteristic`, `PenComponent`, `PenMake`, `WritingImplement`, `Pen`, `BallpointPen`, `FunctionPen`, and `QuillPen`. The 'Individuals' tab is selected, showing 'Direct instances: Fisher' for the class `BallpointPen`. A dialog box titled 'Create a new Named individual' is open, with the 'Name' field set to 'ExampleBallpointPen1' and the 'IRI' field set to 'http://example.com/BallpointPenOntology/#ExampleBallpointPen1'. The 'OK' button is highlighted.

Right Screenshot: The application window shows the same ontology. The 'Class hierarchy' tab is active, and the 'Individuals' tab is selected. The 'Direct instances' list now includes 'ExampleBallpointPen1' for the class `BallpointPen`. The 'Annotations' tab is also visible, showing 'Annotations: ExampleBallpointPen1'.

Add data type to instance

The image displays two screenshots of the Protégé OWL editor interface, illustrating the process of adding a data type to an instance.

Left Screenshot: The "ExampleBallpointPen1" dialog box is open, showing the "Data Property" tab. The "Value" field contains "13.8". The "Datatype" dropdown is set to "xsd:float". The "Language Tag" field is empty. The "OK" button is visible.

Right Screenshot: The main Protégé window shows the "Class hierarchy" for "BallpointPen". The "Individuals" tab is selected, showing "ExampleBallpointPen1" as a direct instance. The "Property assertions" panel on the right shows the assertion for "hasWidestDiameterInMillimeters 13.8f".

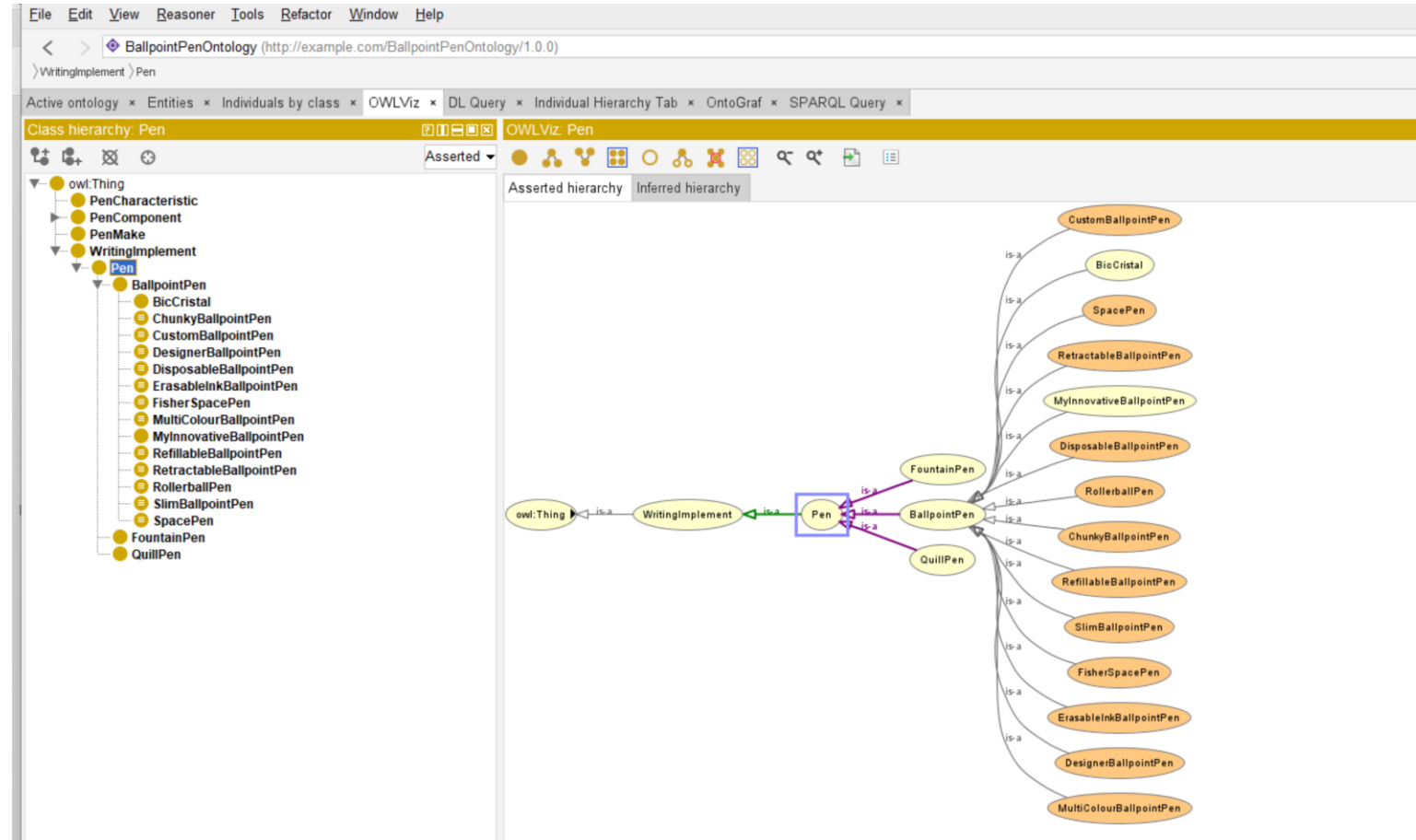
Add a restriction involving the datatype property

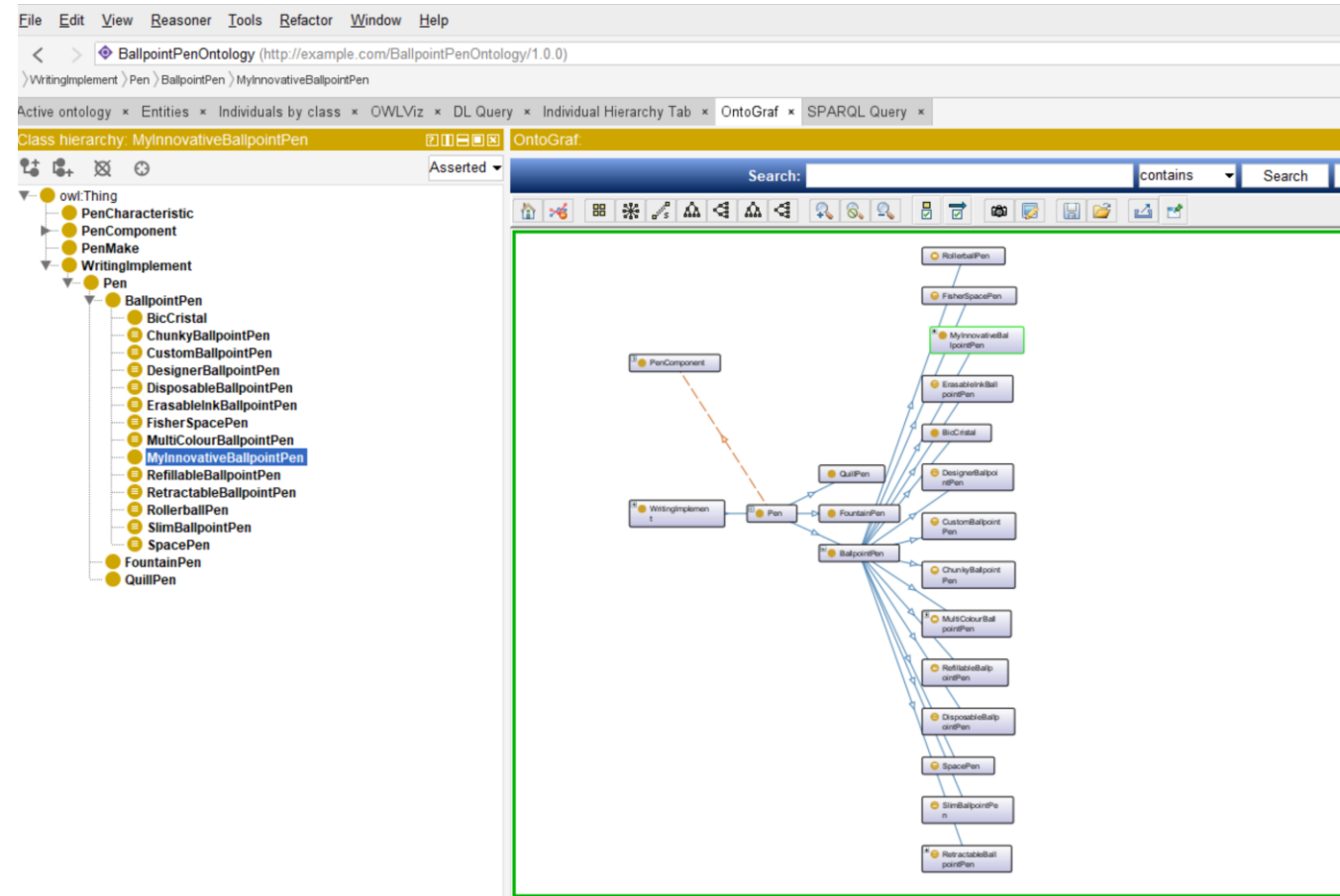
The image displays two screenshots of the Protégé ontology editor interface, illustrating the process of adding a datatype restriction to the `ChunkyBallpointPen` class.

Left Screenshot: The `ChunkyBallpointPen` class is selected in the class hierarchy. The `Annotations` tab is active, showing the `hasWidestDiameterInMillimeters` property. A dialog box titled `ChunkyBallpointPen` is open, showing the `Class expression editor` with the restriction `hasWidestDiameterInMillimeters some xsd:float [>=13.5]`.

Right Screenshot: The `ChunkyBallpointPen` class is selected in the class hierarchy. The `Direct instances (inferred)` tab is active, showing the class description: `Equivalent To: BallpointPen and (hasWidestDiameterInMillimeters some xsd:float [>= 13.5])`. The `SubClass Of` section shows the assertion: `Asserted in: http://example.com/BallpointPenOntology/1.0.0`.

Visualization





deploy

- OWL to HTML
- <https://essepuntato.it/lode/>

- aa